

'26.01.05. ~ 01.22.

하만 세미콘 아카데미 최종 프로젝트

SoC 시스템 기반

펜 플로터를 이용한 커스텀 각인 서비스

3팀 지 - 잉

변지인 김태형 안유한
이윤지 채민석 채준희 한규혁



팀원 소개



변지인 (팀장)
FPGA
Camera IF / UART 설계
전체 시스템 통합



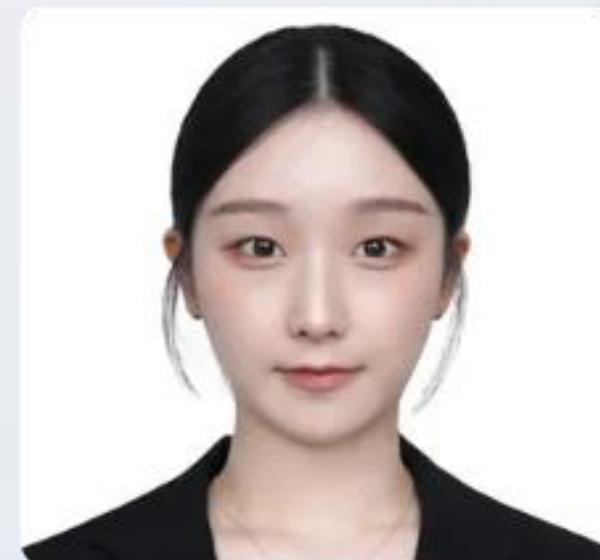
김태형
FPGA
이미지 전처리
이미지 처리 성능 최적화



채민석 (발표자)
FPGA
Canny Edge 구현
이미지 처리 성능 최적화



안유한
임베디드 / Python
STM32 UART/Queue 설계
통합 GUI 환경 구축



이윤지
임베디드
Motor 동작 설계
펜 플로터 제작



채준희
임베디드 / 하드웨어 설계
RTOS 프레임워크 구축
모터 구동 알고리즘 작성



한규혁
임베디드
Motor 2종 / LCD 설계
RTOS Task 동기화 설계

Contents

01
Project Overview

02
FPGA

03
Python

04
STM32

05
Troubleshooting

06
고 찰

01. Project Overview



주제 선정 배경

스타벅스, 이대점 등
커피 커스터마이징 소비 증가



고가 장비

긴 제작 시간

현장 즉시 제공 어려움

소규모 행사·이벤트 적용 한계

주제 선정 배경

스타벅스, 이대점 등
커피 커스터마이징 소비 증가



고가 장비

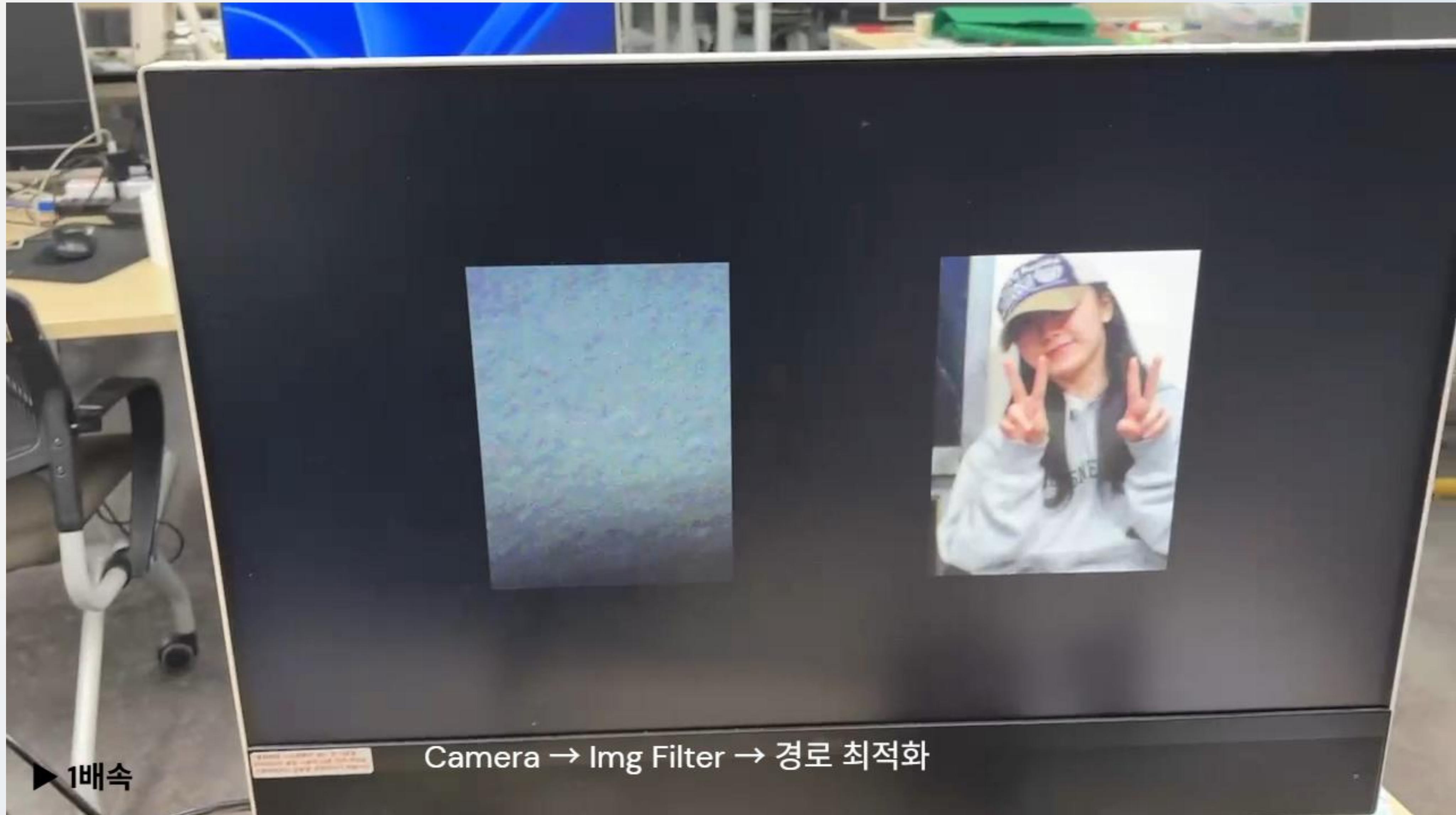
긴 제작 시간

"간편" "저비용" "즉시 제공" 가능한 각인 시스템

증가 어려움

소규모 행사·이벤트 적용 한계

시연 영상



Development Environment

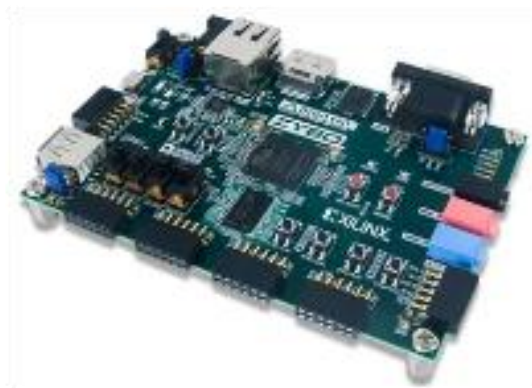
HW



OV7670



STM32



Zybo Z7-20

Language



SystemVerilog



C Language



Python

Tools



Vivado

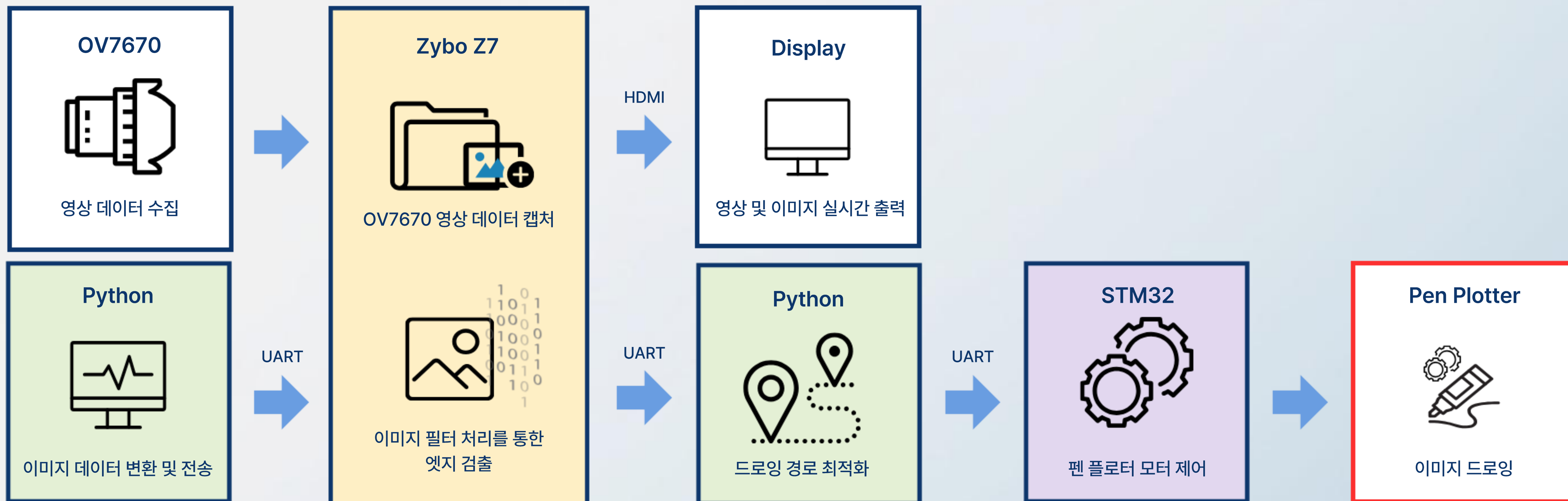


STM32CubeIDE



Visual Studio

System Process



02. FPGA



Edge Detection Algorithm

OV7670 또는 PC로 이미지를 받아 Gray -> Gaussian -> Sobel -> Canny 단계를 통해 외곽선만 도출



Edge Detection Algorithm

OV7670 또는 PC로 이미지를 받아 Gray -> Gaussian -> Sobel -> Canny 단계를 통해 외곽선만 도출



Edge Detection Algorithm

OV7670 또는 PC로 이미지를 받아 Gray -> Gaussian -> Sobel -> Canny 단계를 통해 외곽선만 도출



Edge Detection Algorithm

OV7670 또는 PC로 이미지를 받아 Gray -> Gaussian -> Sobel -> Canny 단계를 통해 외곽선만 도출



Edge Detection Algorithm

OV7670 또는 PC로 이미지를 받아 Gray -> Gaussian -> Sobel -> Canny 단계를 통해 외곽선만 도출



Sobel Filter

Sobel Filter

밝기 변화량 검출 → 경계선 정보 추출

-1	-2	-1
0	0	0
1	2	1

세로 방향

-1	0	1
-2	0	2
-1	0	1

가로 방향

1. Gradient 크기 연산

- 픽셀 밝기의 급격한 변화량을 수직, 수평방향으로 계산
- FPGA 내부에서 구현이 복잡한 계산식(제곱근) 대신 절댓값 사용

$$|G| \approx |G_x| + |G_y|$$

2. 포화 처리 (Saturation)

결과값이 8 bit 범위 초과 시, 최댓값(255)으로 고정 → 이미지 왜곡 방지

$$G_{out} = (G > 255) ? 255 : G[7 : 0]$$

Sobel Filter

Sobel Filter

밝기 변화량 검출 → 경계선 정보 추출

-1	-2	-1
0	0	0
1	2	1

세로 방향

-1	0	1
-2	0	2
-1	0	1

가로 방향

1. Gradient 크기 연산

- 픽셀 밝기의 급격한 변화량을 수직, 수평방향으로 계산
- FPGA 내부에서 구현이 복잡한 계산식(제곱근) 대신 절댓값 사용

$$|G| \approx |G_x| + |G_y|$$

2. 포화 처리 (Saturation)

결과값이 8 bit 범위 초과 시, 최댓값(255)으로 고정 → 이미지 왜곡 방지

$$G_{out} = (G > 255) ? 255 : G[7 : 0]$$

Sobel Filter

Sobel Filter

밝기 변화량 검출 → 경계선 정보 추출

-1	-2	-1
0	0	0
1	2	1

세로 방향

-1	0	1
-2	0	2
-1	0	1

가로 방향

1. Gradient 크기 연산

- 픽셀 밝기의 급격한 변화량을 수직, 수평방향으로 계산
- FPGA 내부에서 구현이 복잡한 계산식(제곱근) 대신 절댓값 사용

$$|G| \approx |G_x| + |G_y|$$

2. 포화 처리 (Saturation)

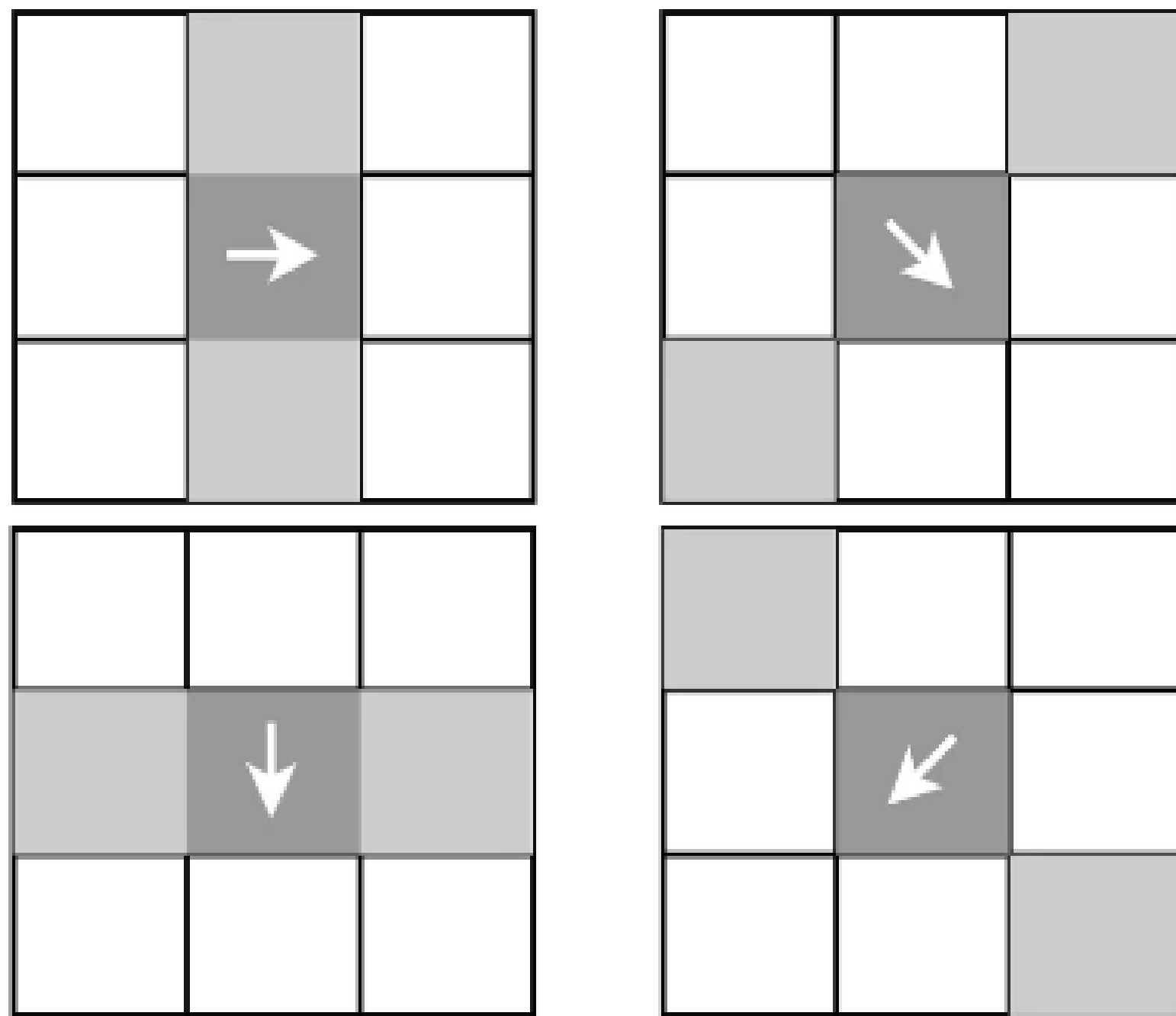
결과값이 8 bit 범위 초과 시, 최댓값(255)으로 고정 → 이미지 왜곡 방지

$$G_{out} = (G > 255) ? 255 : G[7 : 0]$$

Canny Edge

Canny Edge

불필요한 엣지 제거 → 정밀 윤곽선 추출



1. 비최대값 억제 (NMS)

- 뭉툭한 선을 1-pixel 두께로 압축
- 변화율이 가장 진한 중심점만 유지, 나머지는 0(Black) 처리

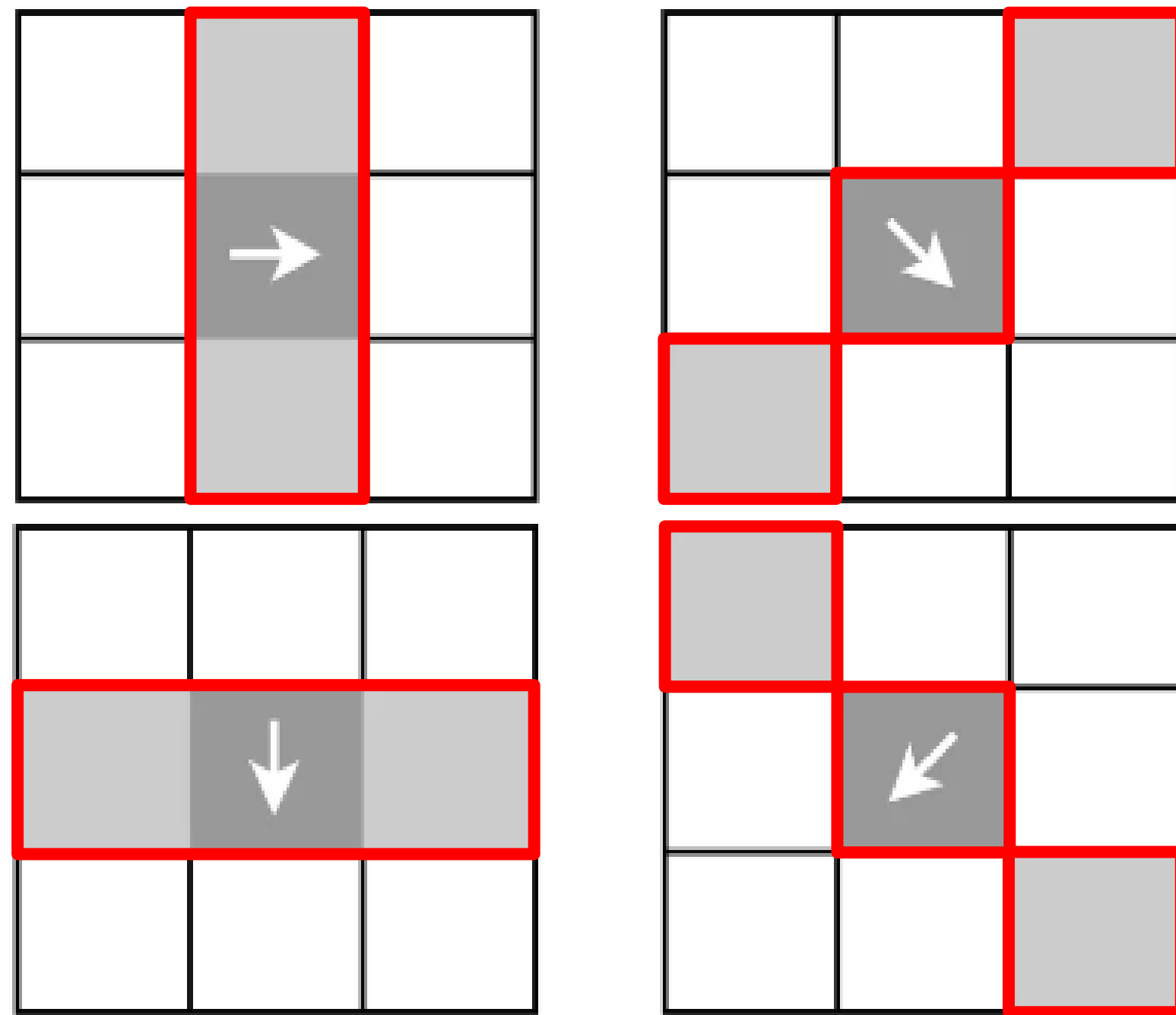
2. 이중 임계값 적용

- T(High), T(Low) 기준 설정을 통한 유효 데이터 선별
- Strong 엣지 이상의 데이터만 최종 엣지로 확정

Canny Edge

Canny Edge

불필요한 엣지 제거 → 정밀 윤곽선 추출



1. 비최댓값 억제 (NMS)

- 뭉툭한 선을 1-pixel 두께로 압축
- 변화율이 가장 진한 중심점만 유지, 나머지는 0(Black) 처리

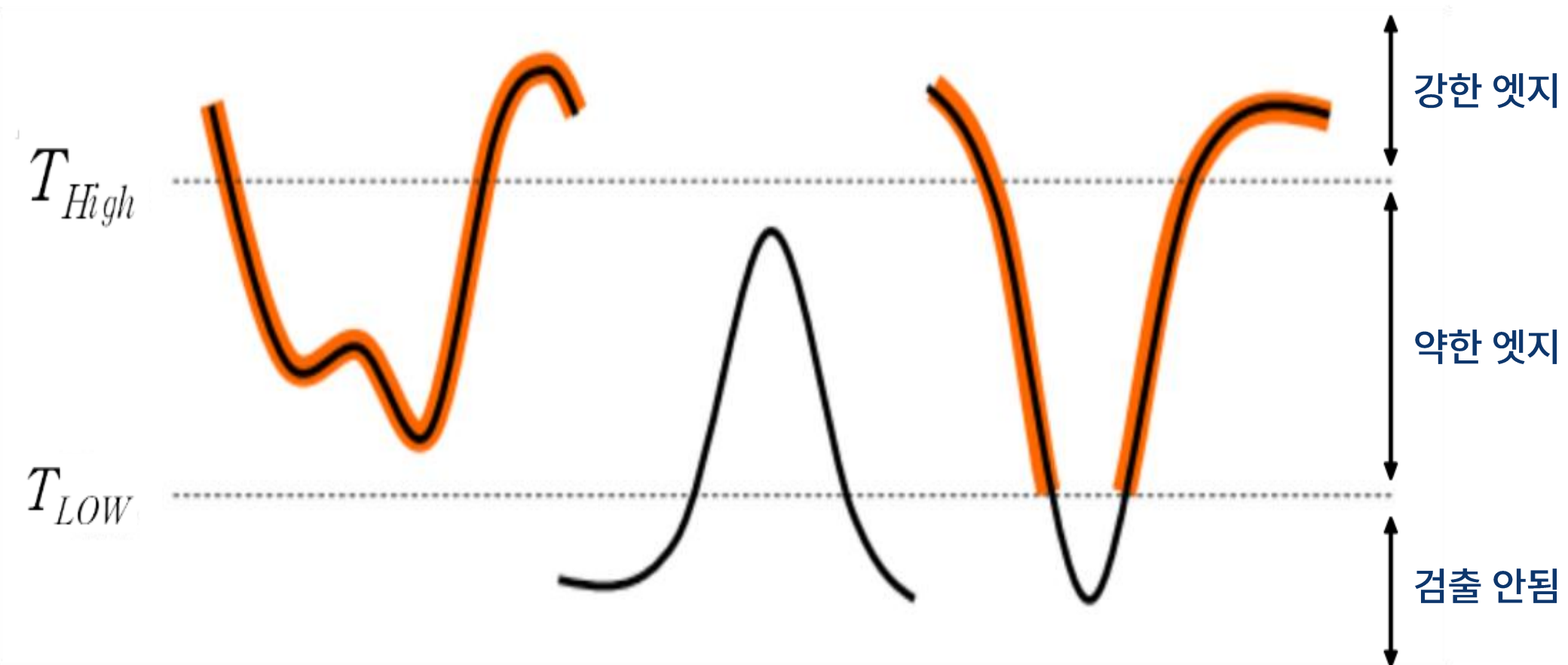
2. 이중 임계값 적용

- T(High), T(Low) 기준 설정을 통한 유효 데이터 선별
- 강한 엣지는 표시, 약한 엣지는 인접한 강한 엣지가 있는 경우 표시

Canny Edge

Canny Edge

불필요한 엣지 제거 → 정밀 윤곽선 추출



1. 비최대값 억제 (NMS)

- 뭉툭한 선을 1-pixel 두께로 압축
- 변화율이 가장 진한 중심점만 유지, 나머지는 0(Black) 처리

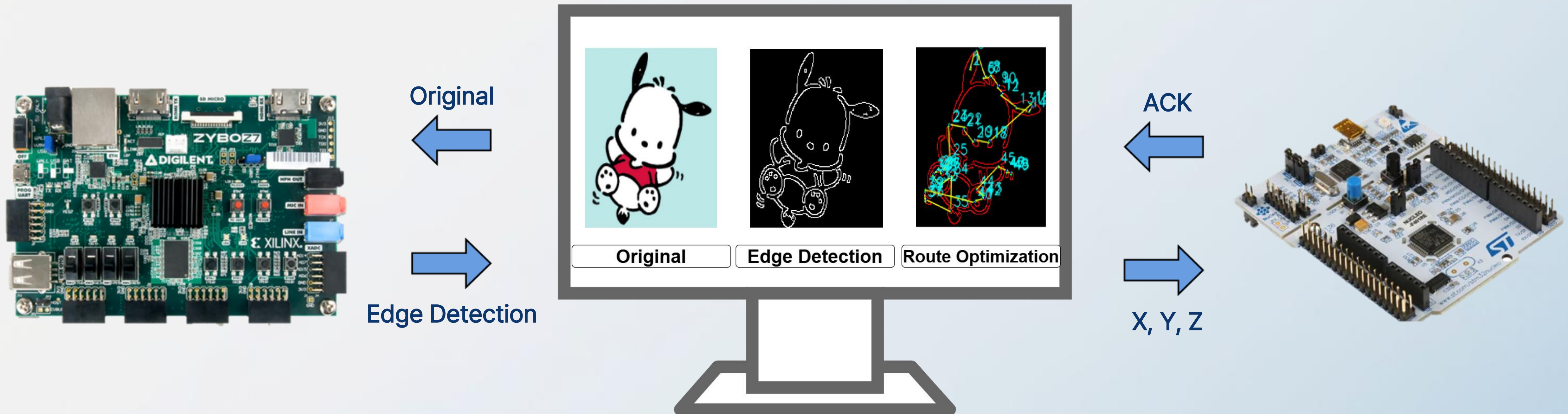
2. 이중 임계값 적용

- T_{High} , T_{Low} 기준 설정을 통한 유효 데이터 선별
- 강한 엣지는 표시, 약한 엣지는 인접한 강한 엣지가 있는 경우 표시

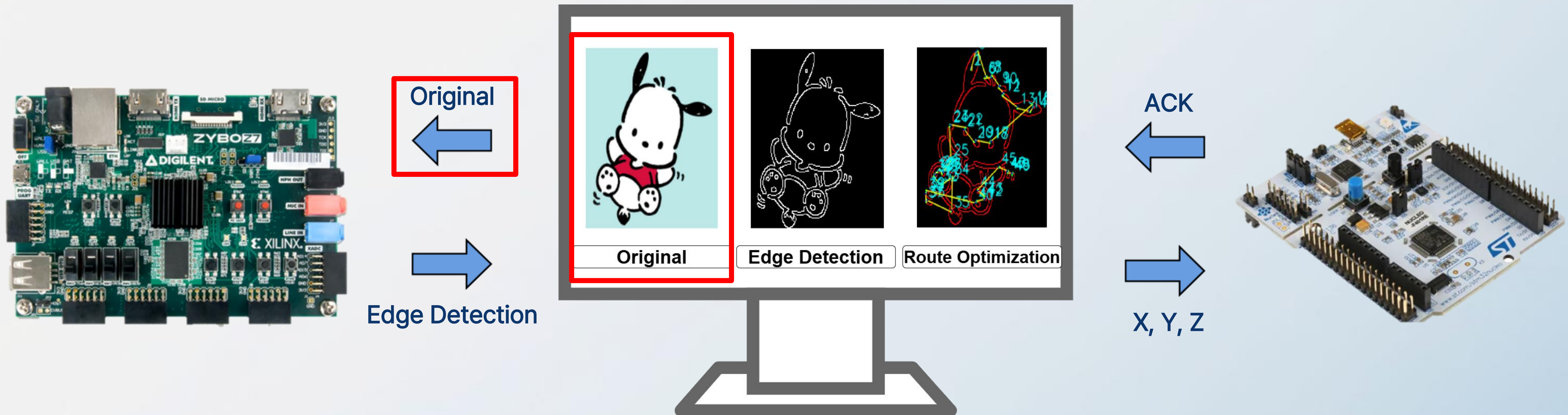
03. Python



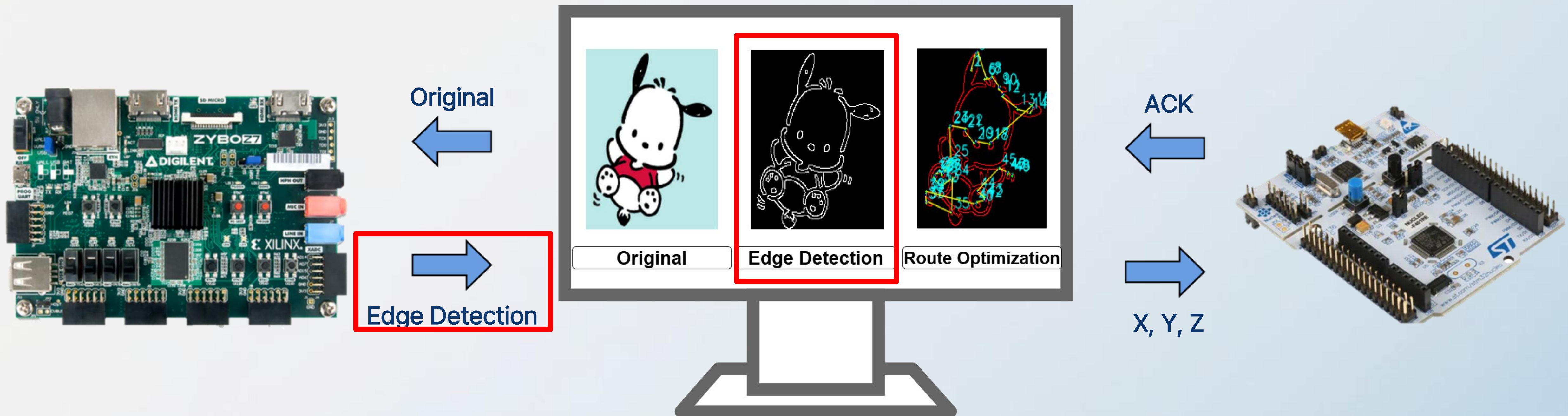
System Architecture



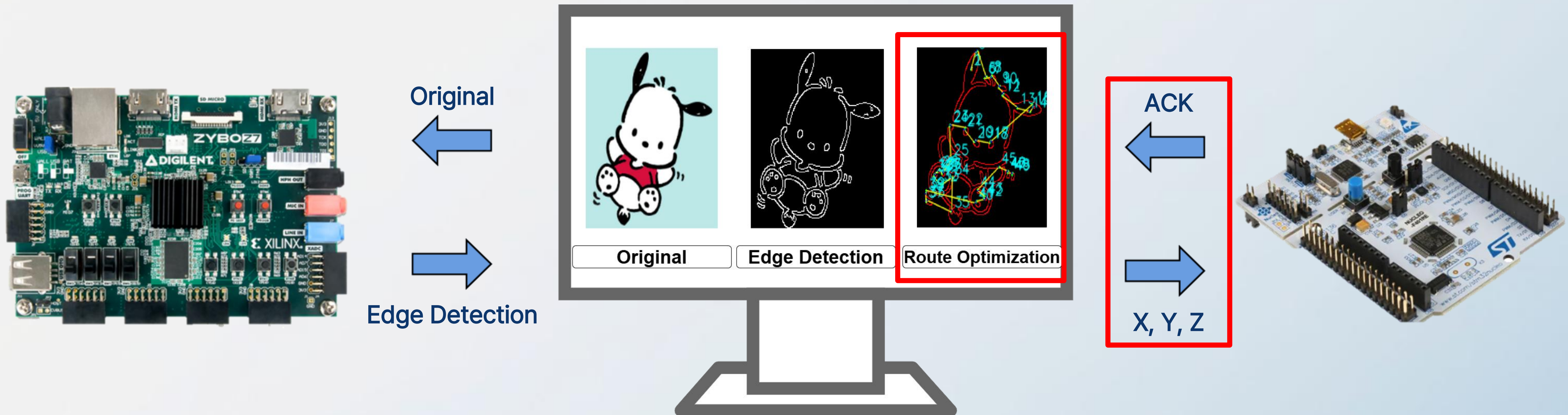
System Architecture



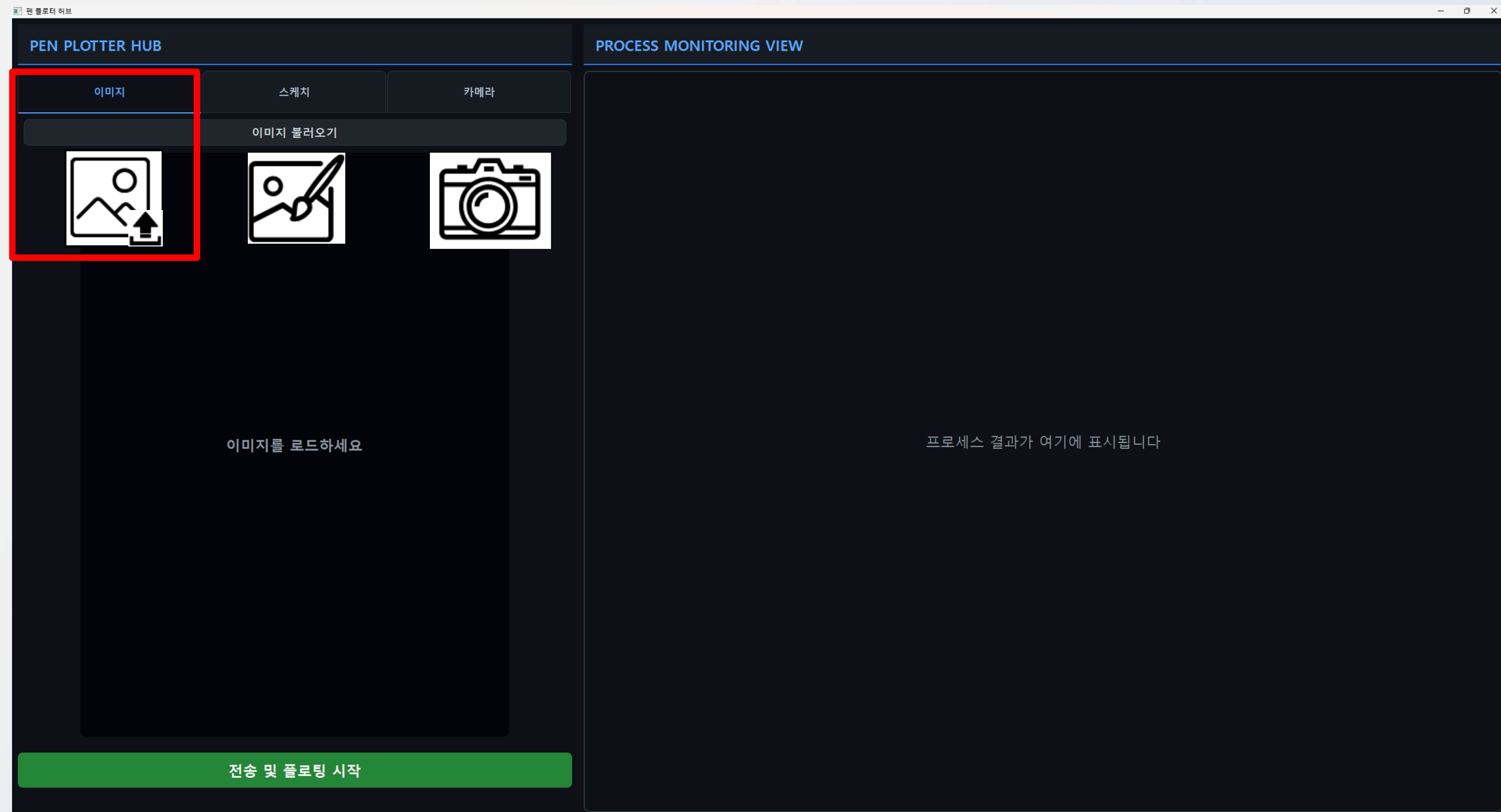
System Architecture



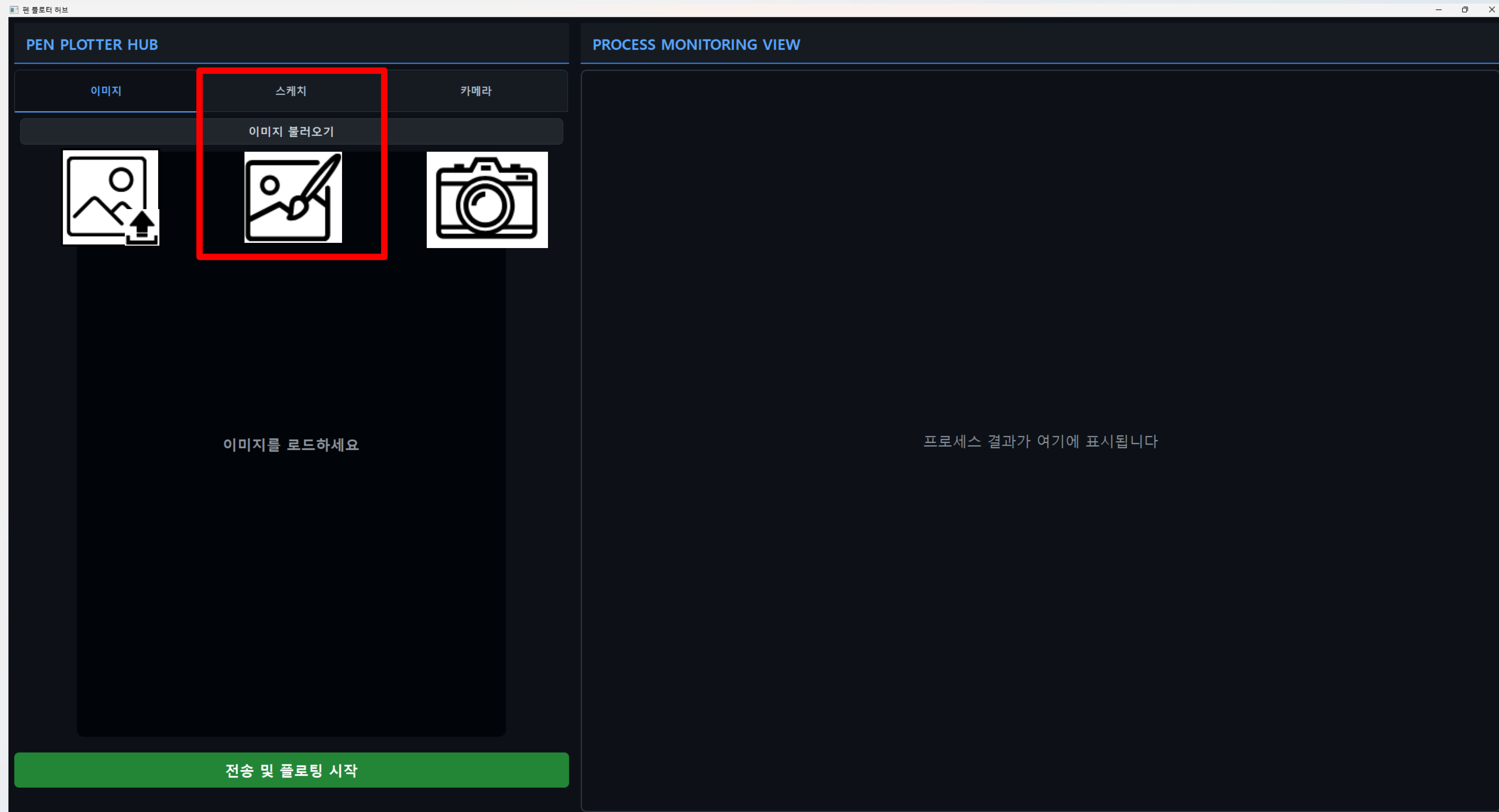
System Architecture



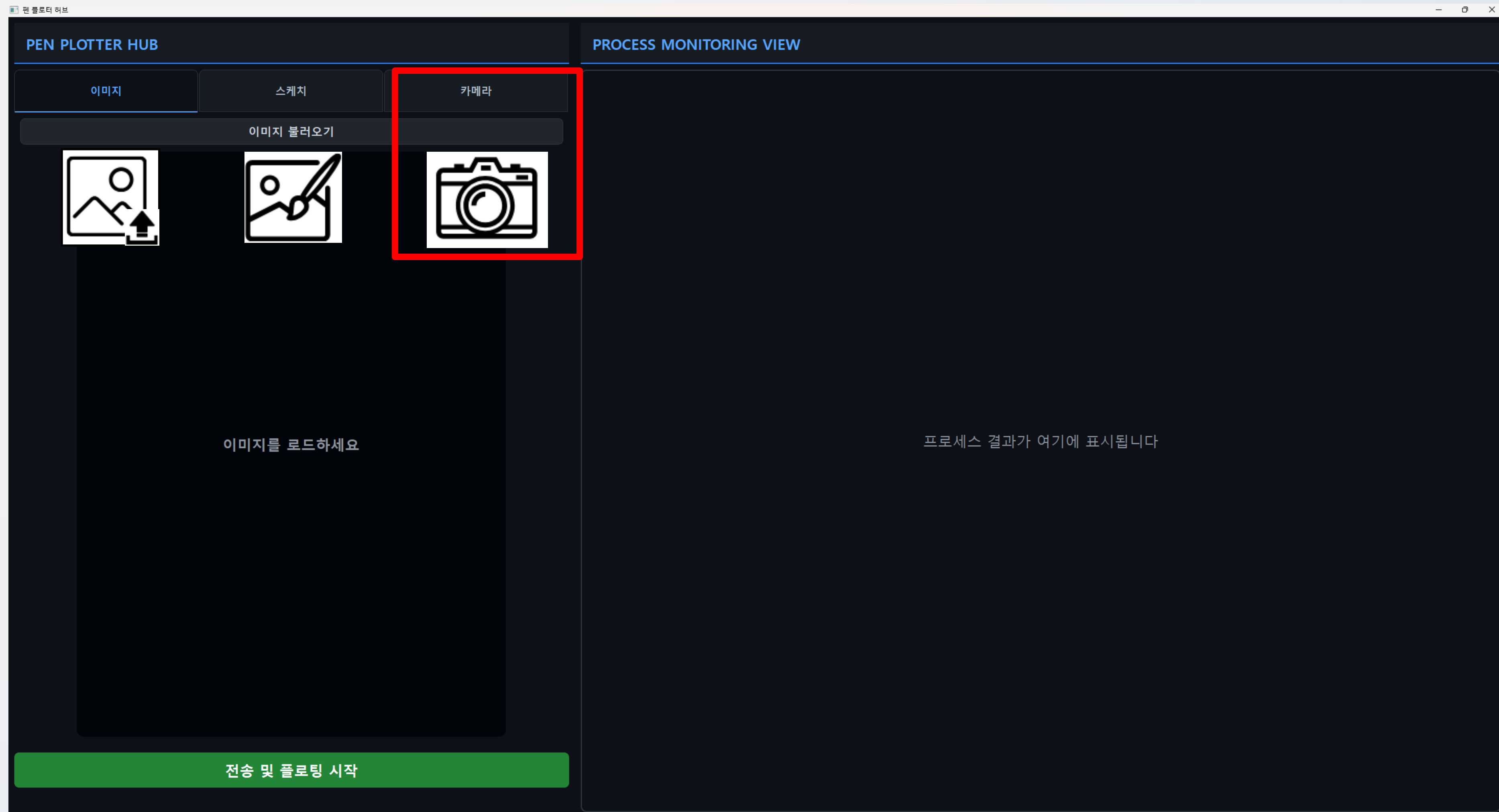
입력 모드 설정 및 결과 모니터링



입력 모드 설정 및 결과 모니터링

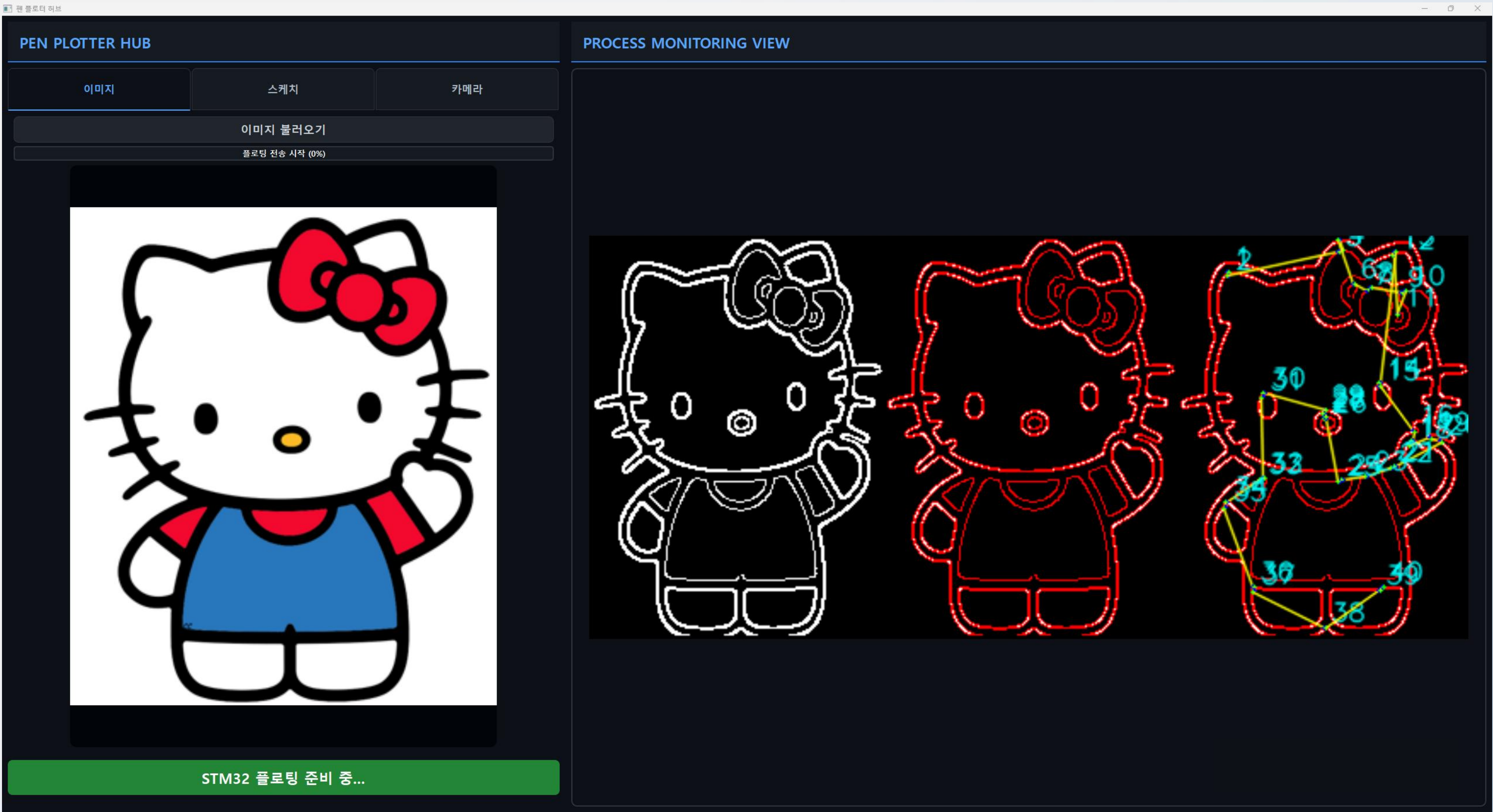


입력 모드 설정 및 결과 모니터링



GUI

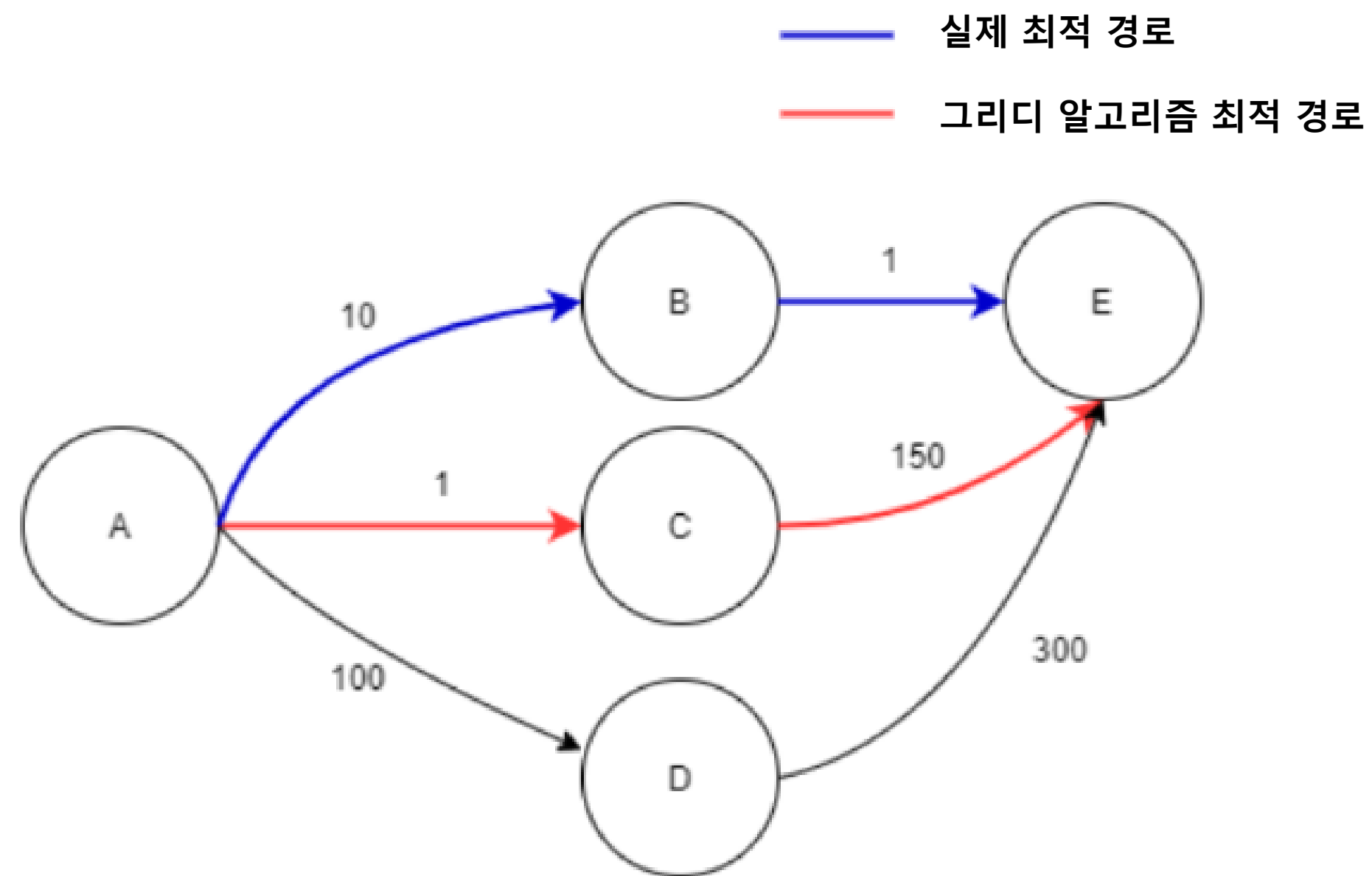
입력 모드 설정 및 결과 모니터링



Route Optimization

Greedy Algorithm

Pen-Up 구간 최소화 → 출력 시간 단축

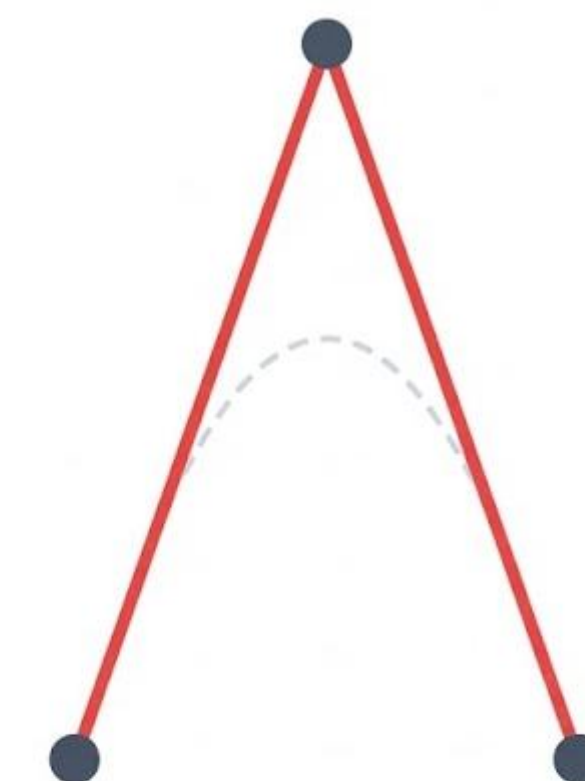


Coordinate Scaling

좌표 간격 세분화 → 부드러운 곡선 보간

Pixel 데이터 (Coarse)

물리 데이터 (Smooth mm)



Densify



각진 경로 (진동 유발)

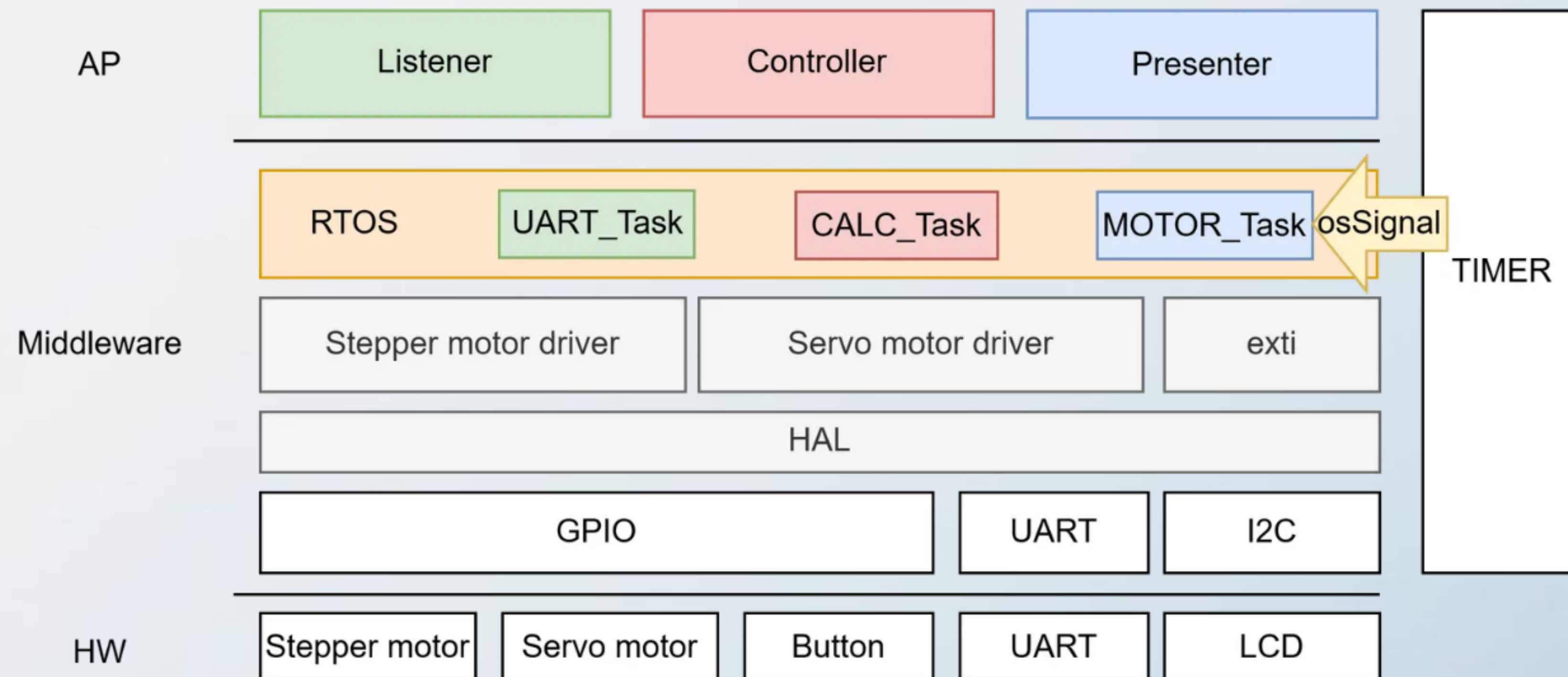
부드러운 곡선 보간

04. STM32



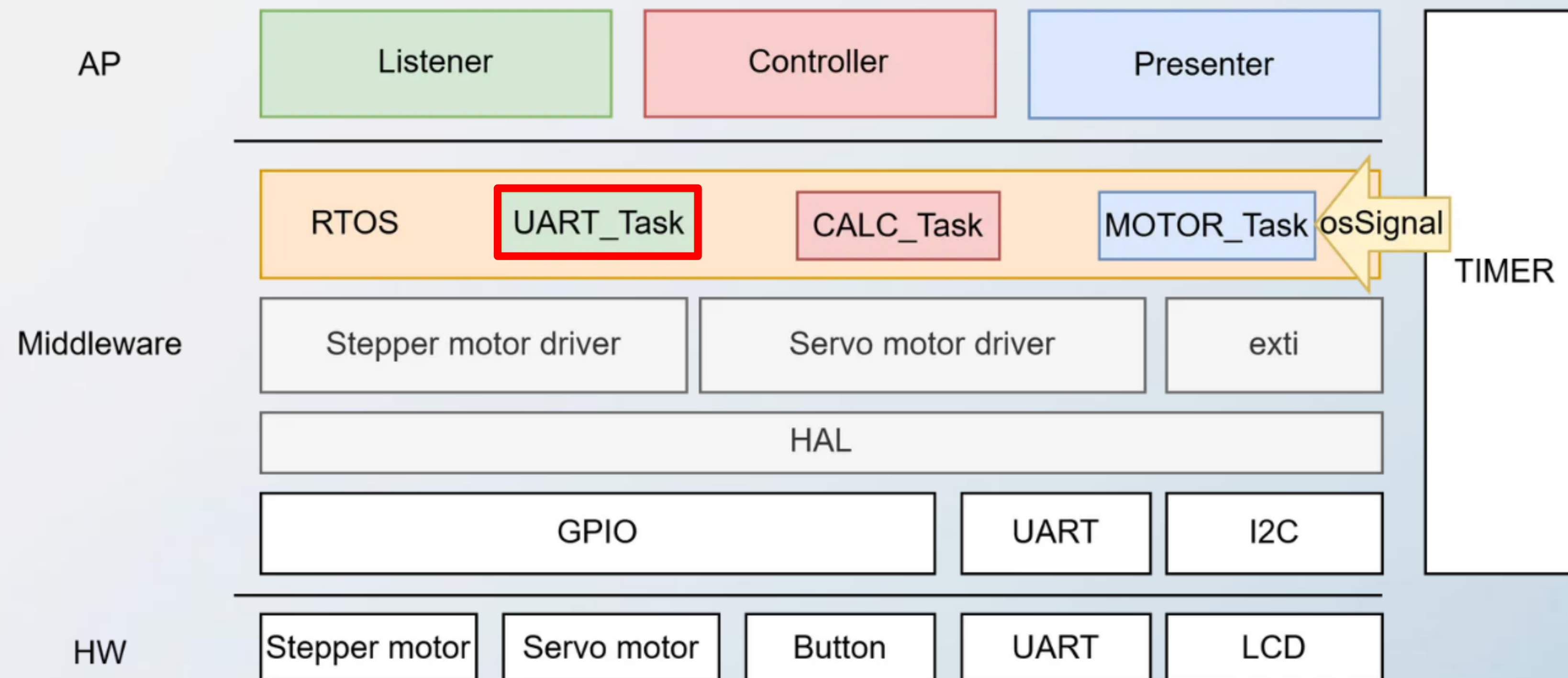
Block Diagram

- **UART_Task** : PC와 UART 통신 담당 Task
- **CALC_Task** : 플로터의 속도, 움직임 계산 Task
- **MOTOR_Task** : 모터를 실제로 구동시키는 Task



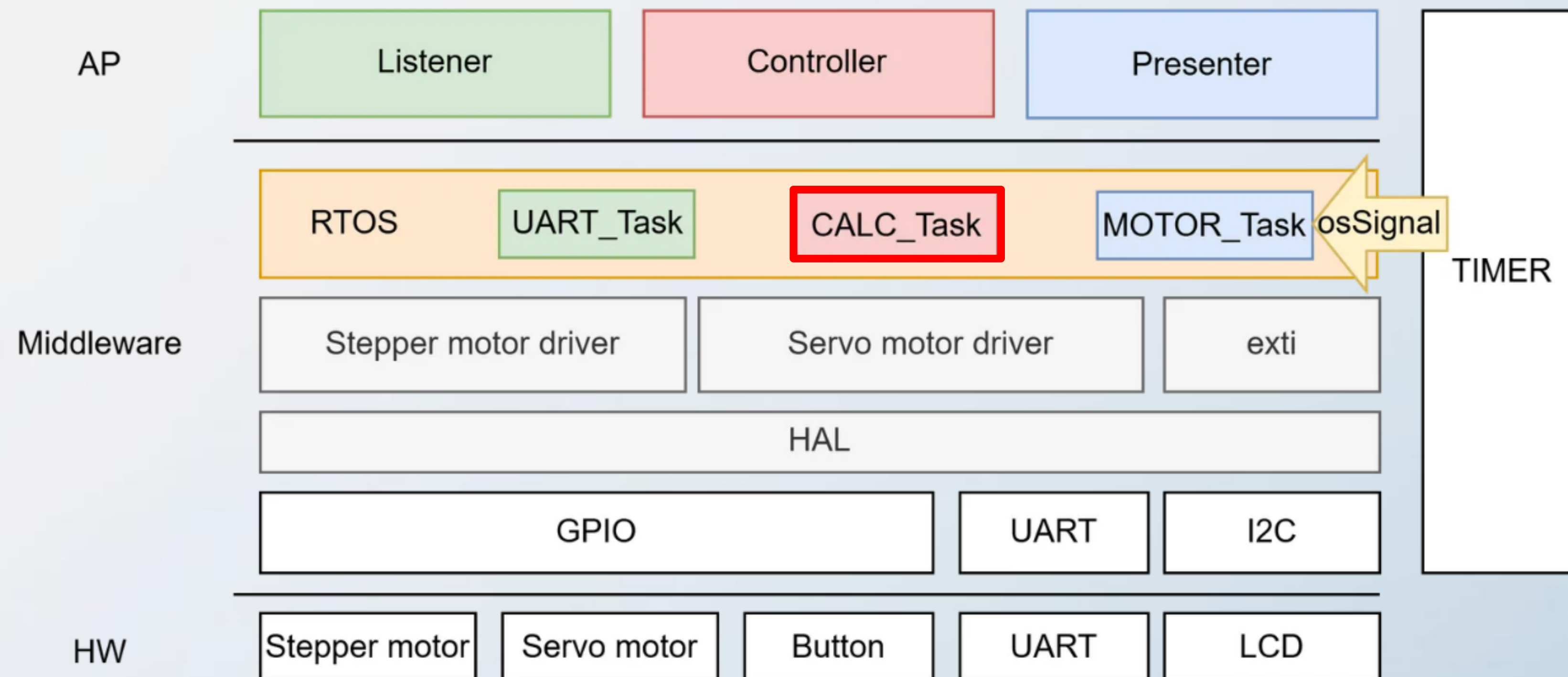
Block Diagram

- **UART_Task** : PC와 UART 통신 담당 Task
- **CALC_Task** : 플로터의 속도, 움직임 계산 Task
- **MOTOR_Task** : 모터를 실제로 구동시키는 Task



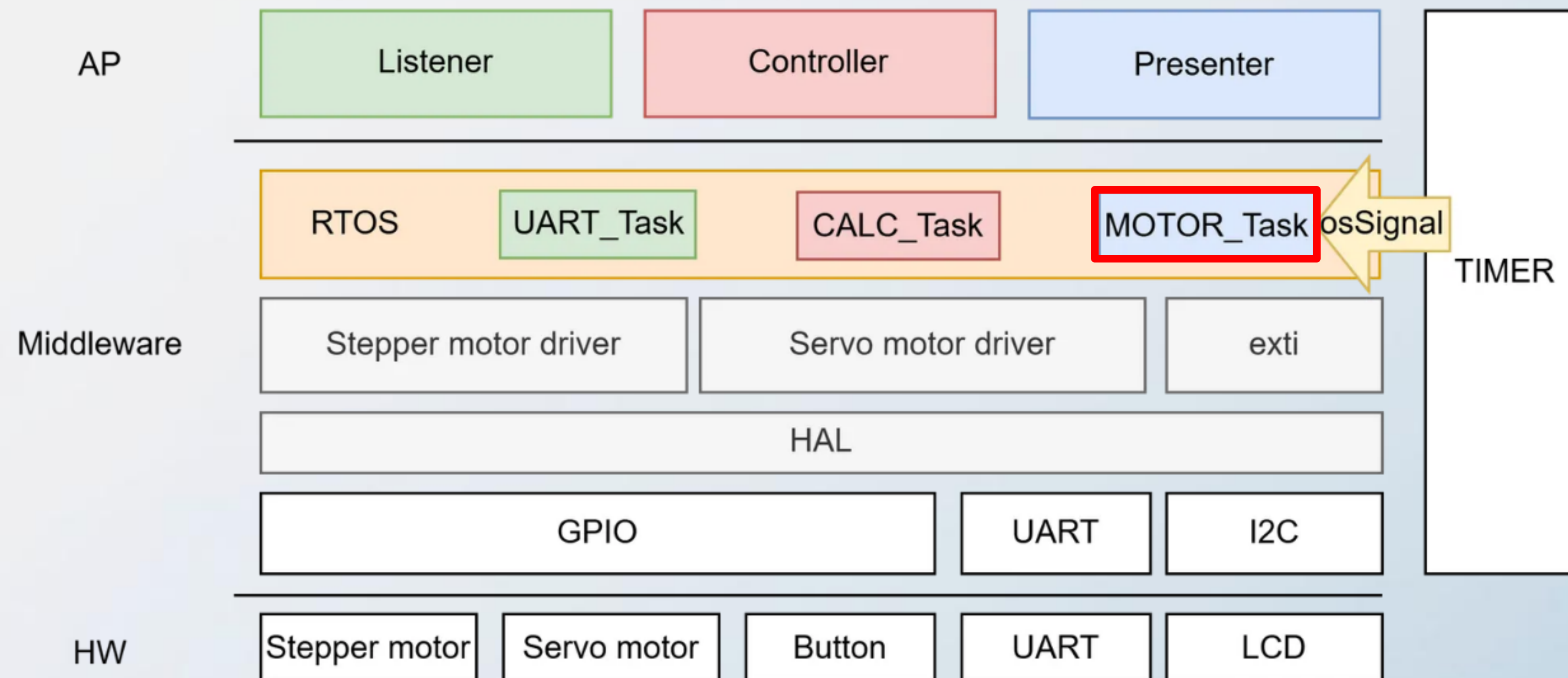
Block Diagram

- **UART_Task** : PC와 UART 통신 담당 Task
- **CALC_Task** : 플로터의 속도, 움직임 계산 Task
- **MOTOR_Task** : 모터를 실제로 구동시키는 Task



Block Diagram

- **UART_Task** : PC와 UART 통신 담당 Task
- **CALC_Task** : 플로터의 속도, 움직임 계산 Task
- **MOTOR_Task** : 모터를 실제로 구동시키는 Task



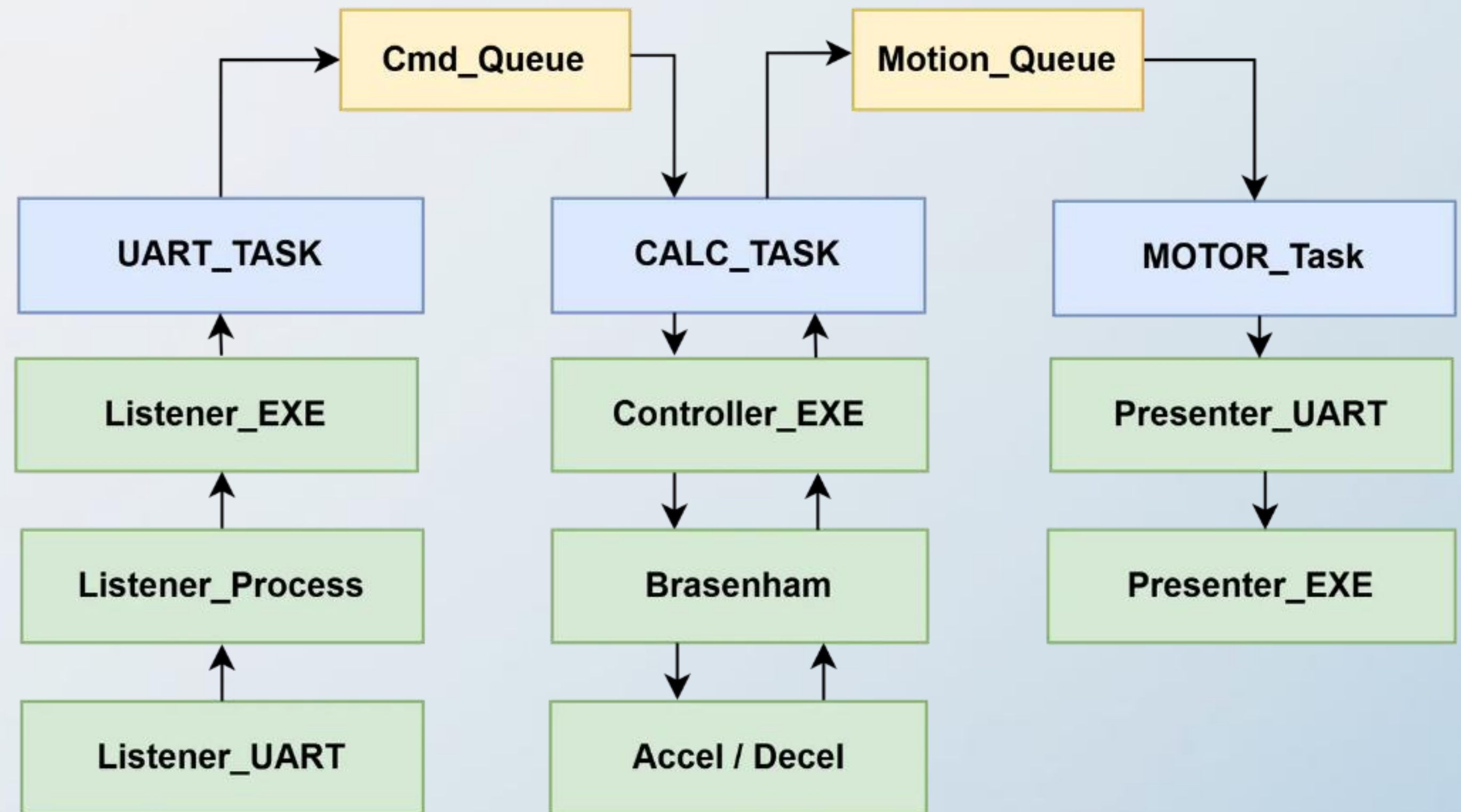
FreeRTOS

Why RTOS?

다중 작업 병렬 처리와 실시간성 보장을 통해 통신 중에도 끊김 없는 정밀한 모터 제어를 구현하기 위함

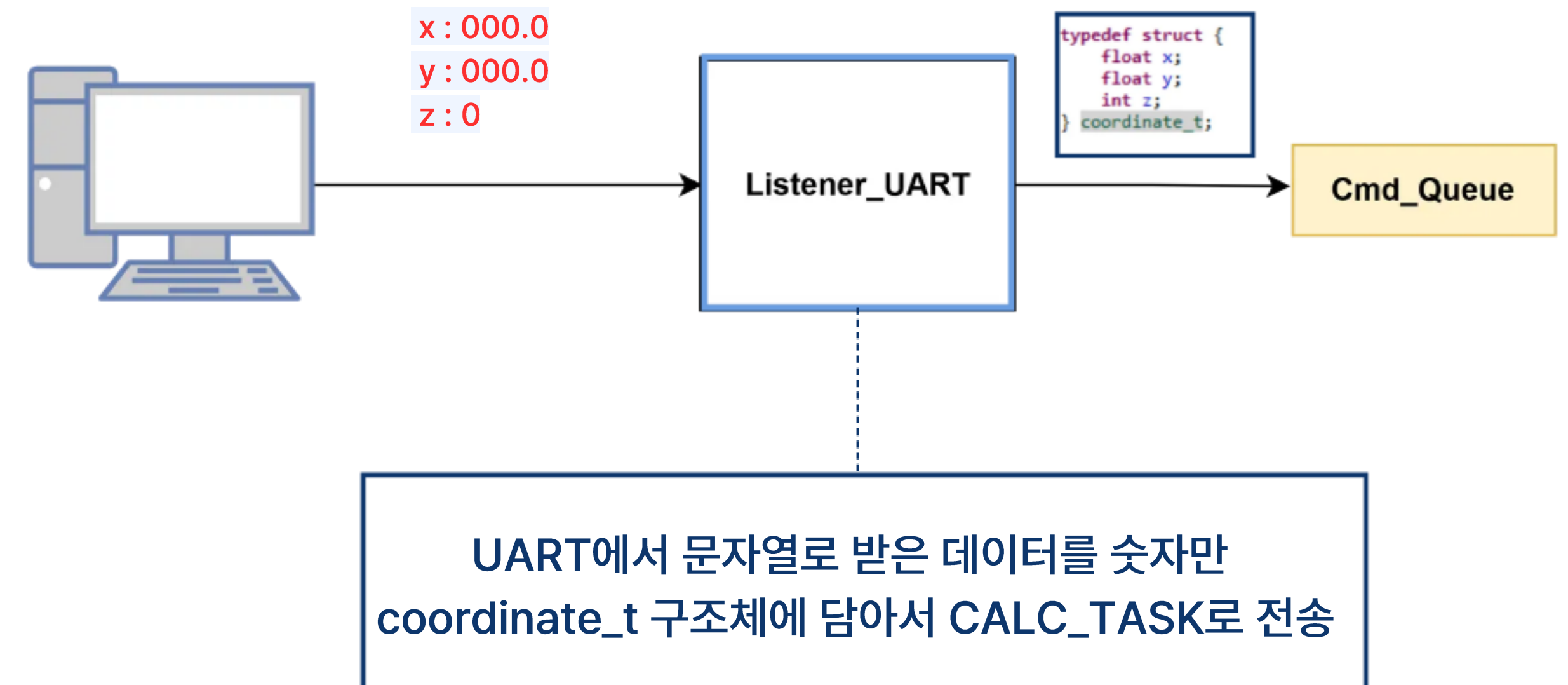
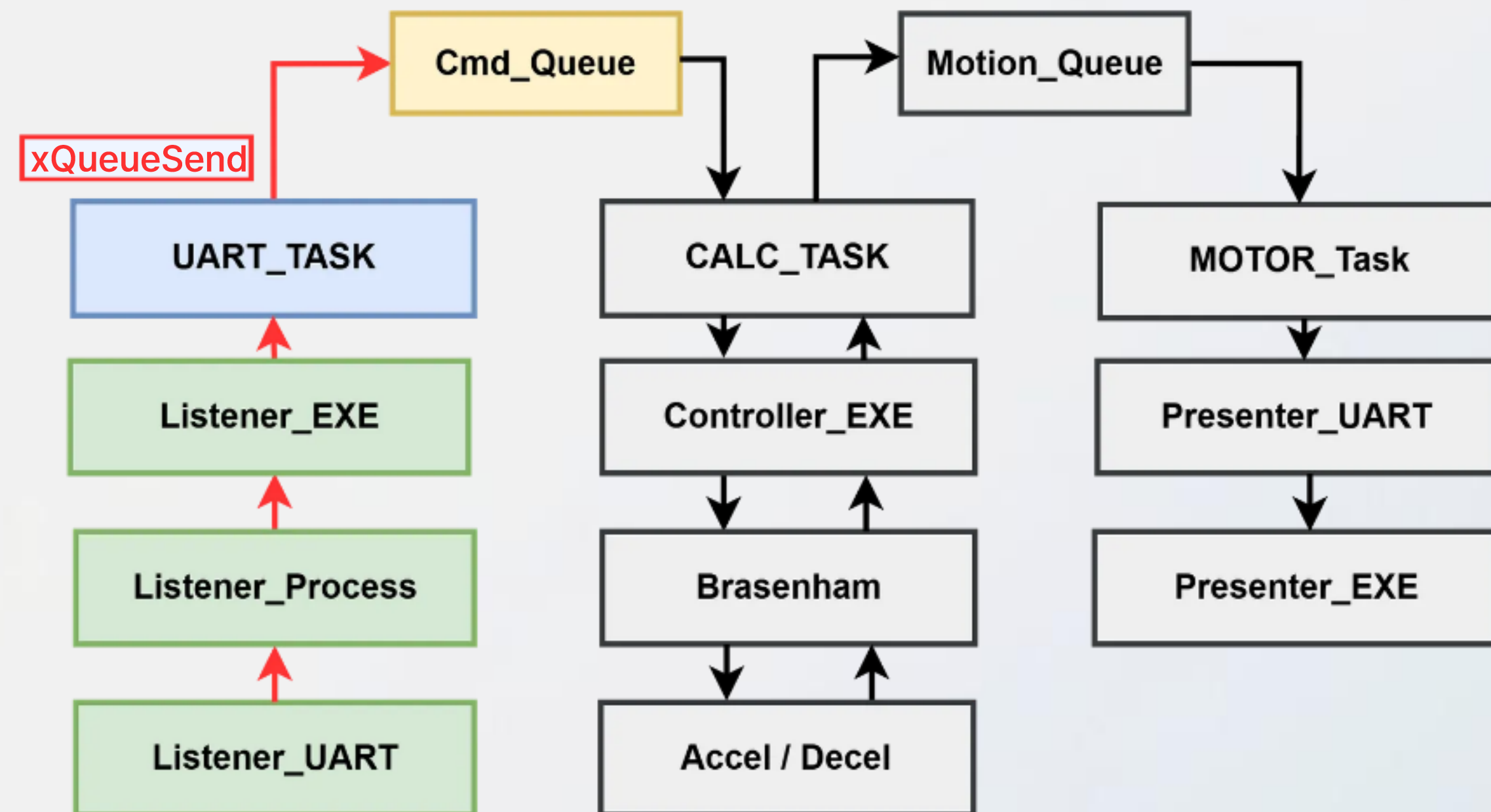
RTOS의 장점

- Listener, Controller, Presenter 분리
→ 코드의 유지보수 편의성 향상
- 전역 변수 없이 Queue 활용
→ Task간 데이터 손실 없이 안전하게 전달
- 여러 작업을 동시 관리
→ 센서 처리, 모터제어, 통신을 실시간으로 수행 가능



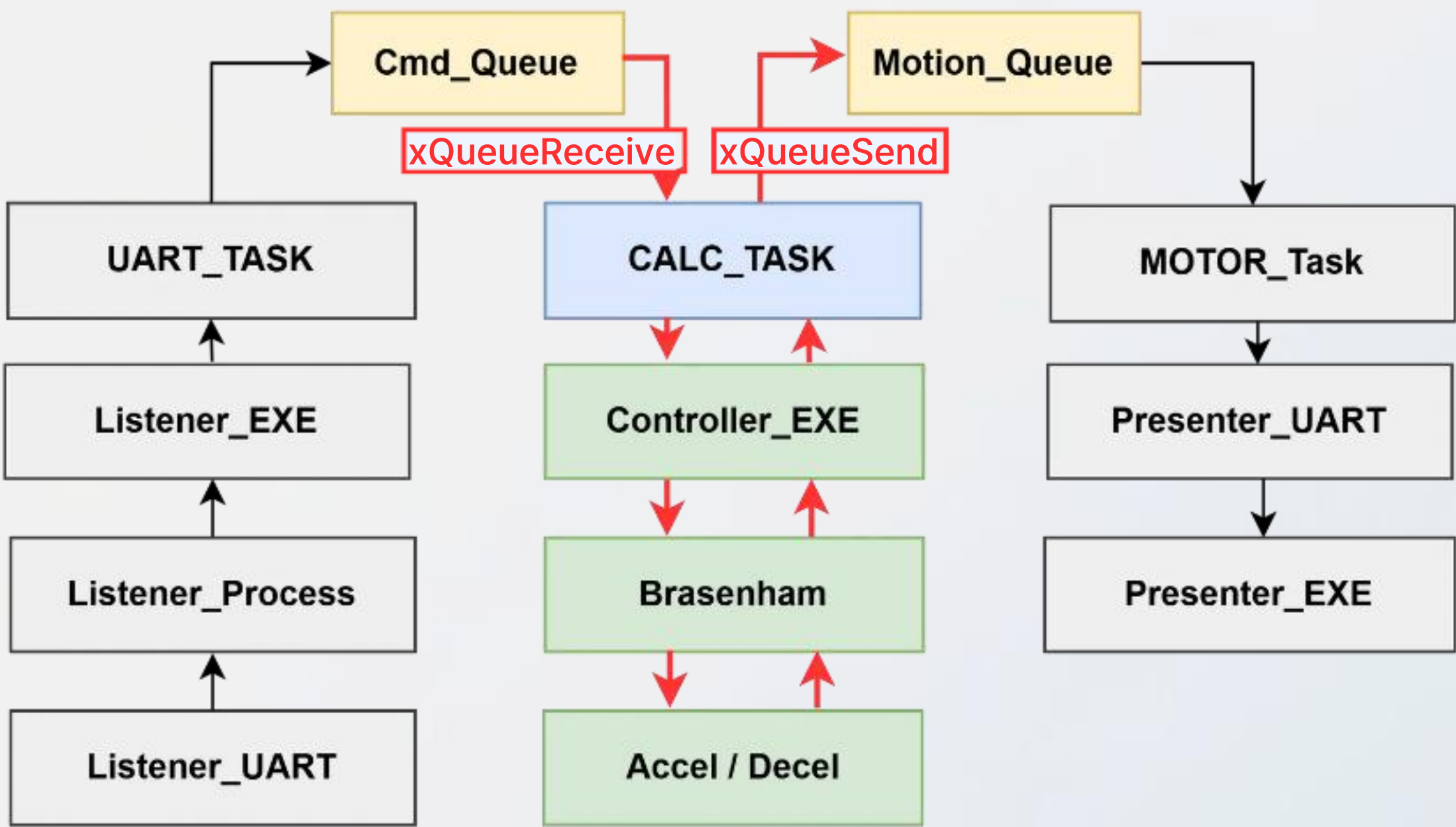
UART_TASK

PC ↔ UART 통신 담당 Task

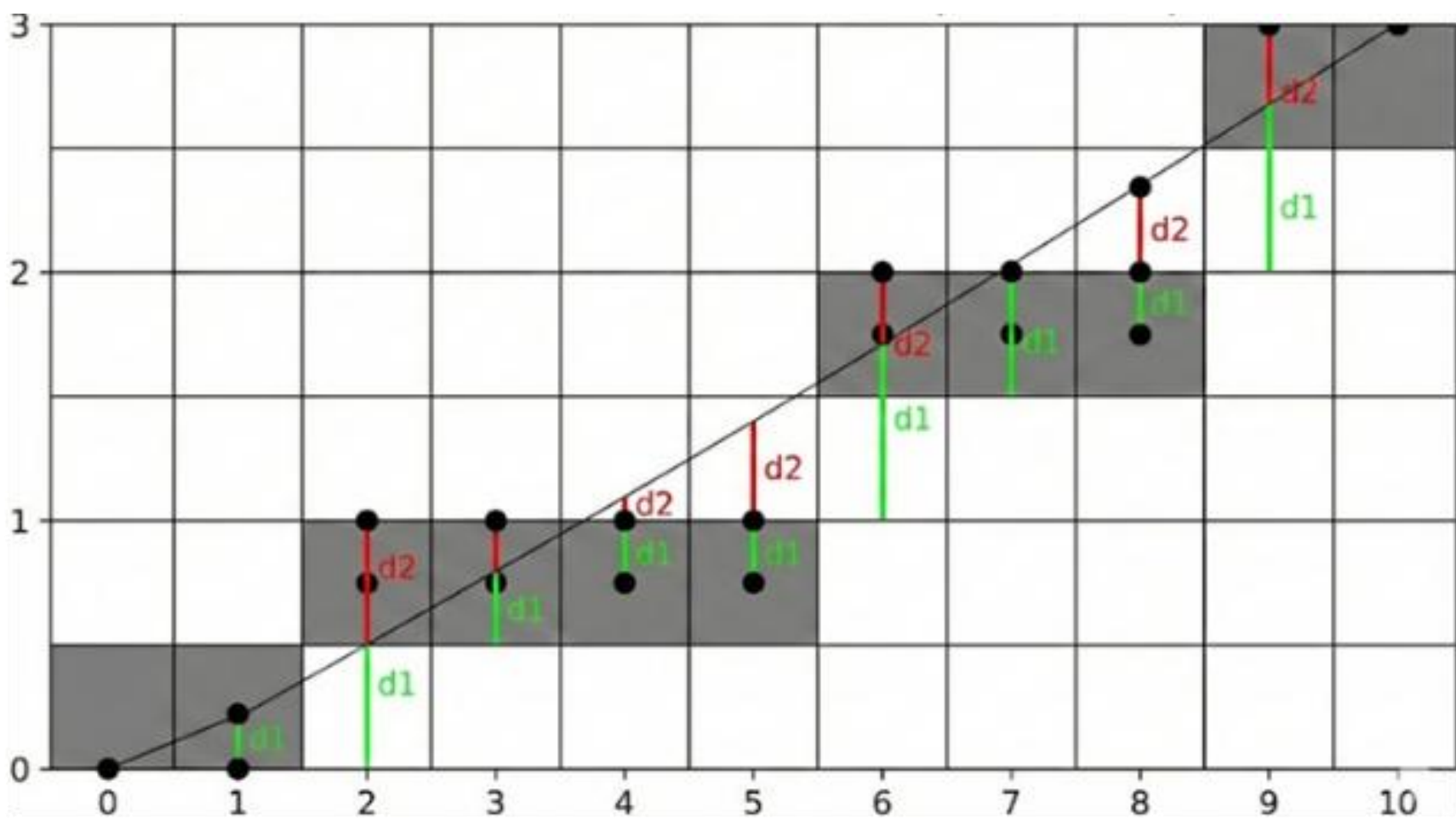


CALC_TASK

플로터의 속도, 움직임 계산 Task



Bresenham 알고리즘



구간	error 변화	보조축 이동	의미
Start	5	-	초기 여유분 설정
1 ~ 2	5 → -1	YES(1)	오차누적 → 첫 보정
3 ~ 5	9 → 0	NO	여유 소진 구간
6	0 → -3	YES (1)	두 번째 보정
7 ~ 9	7 → -2	YES (1)	마지막 보정
End	5	-	초기값 복귀 (오차 0)

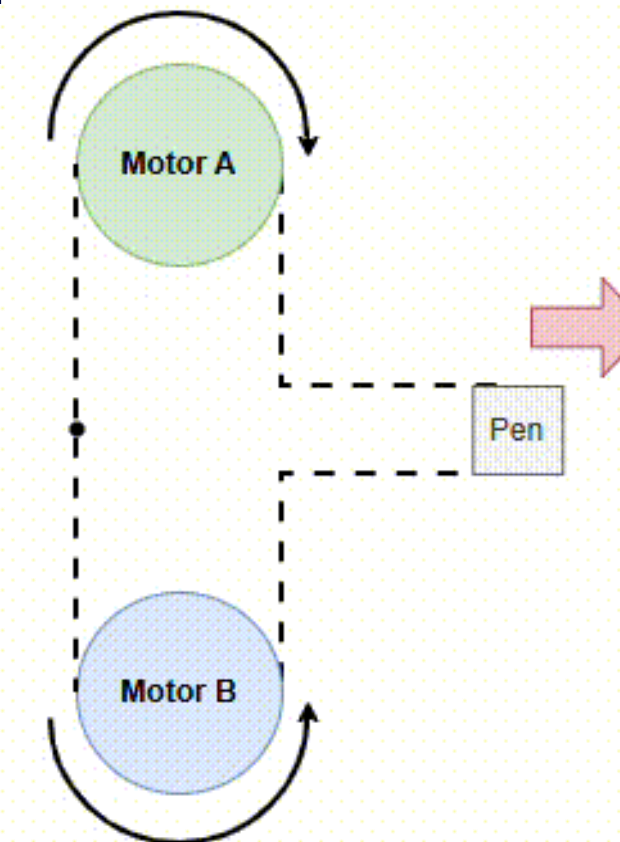
CALC_TASK

플로터의 속도, 움직임 계산 Task

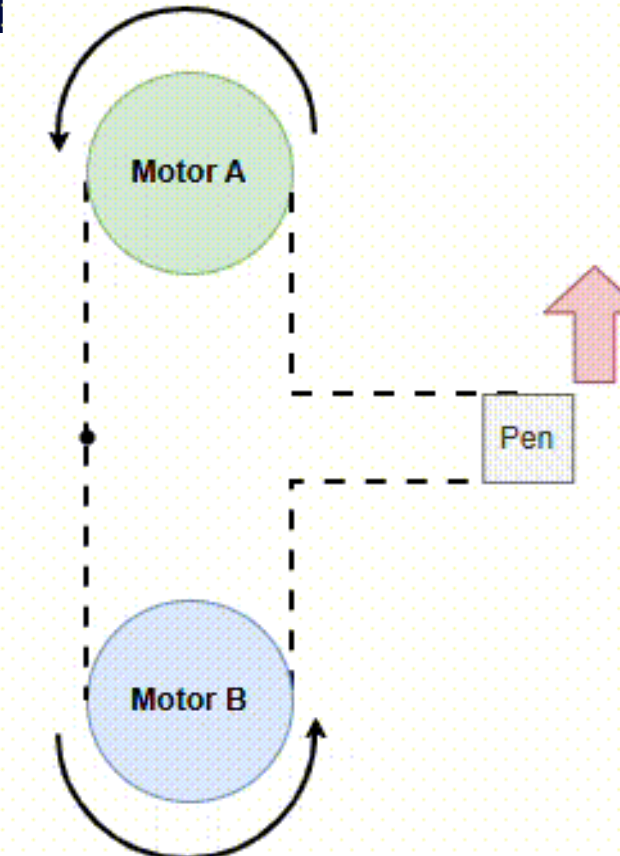
	CW	CCW
CW	Y 감소 ↓	X증가 →
CCW	X 감소 ←	Y 증가 ↑

X,Y가 움직일 **절대값 계산** 후
절댓값이 큰 모터를 주 모터, 작은 것을 보조 모터로 설정

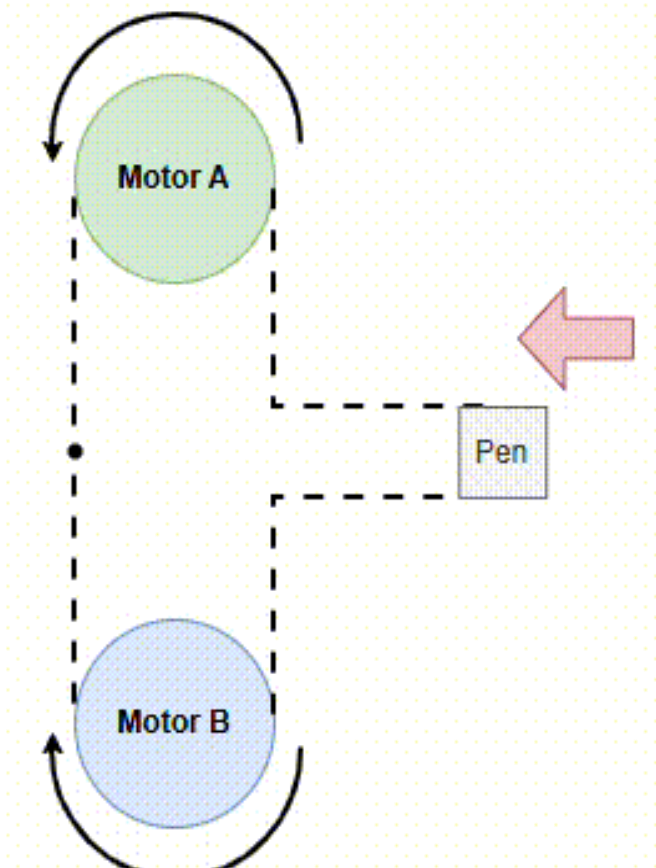
X 증가



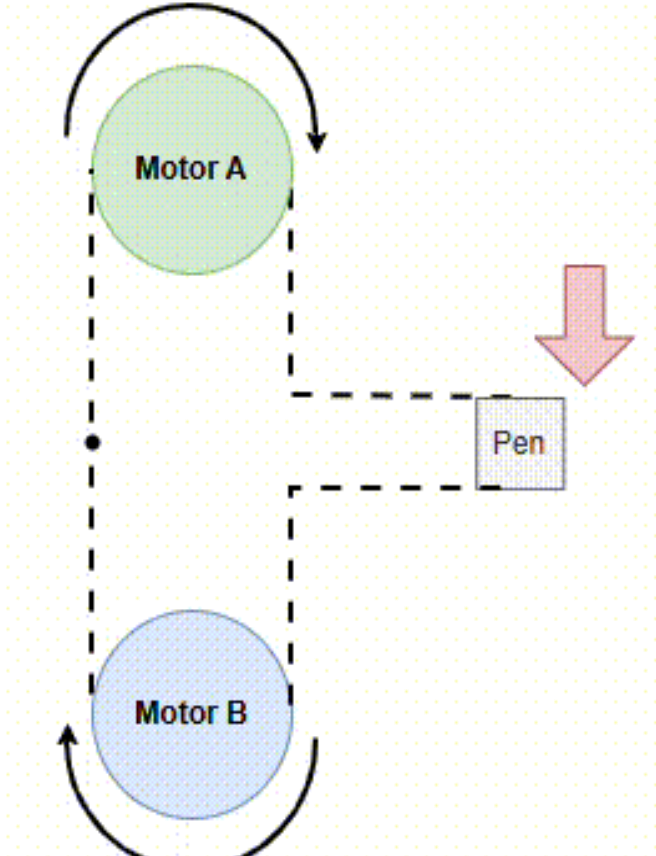
Y 증가



X 감소

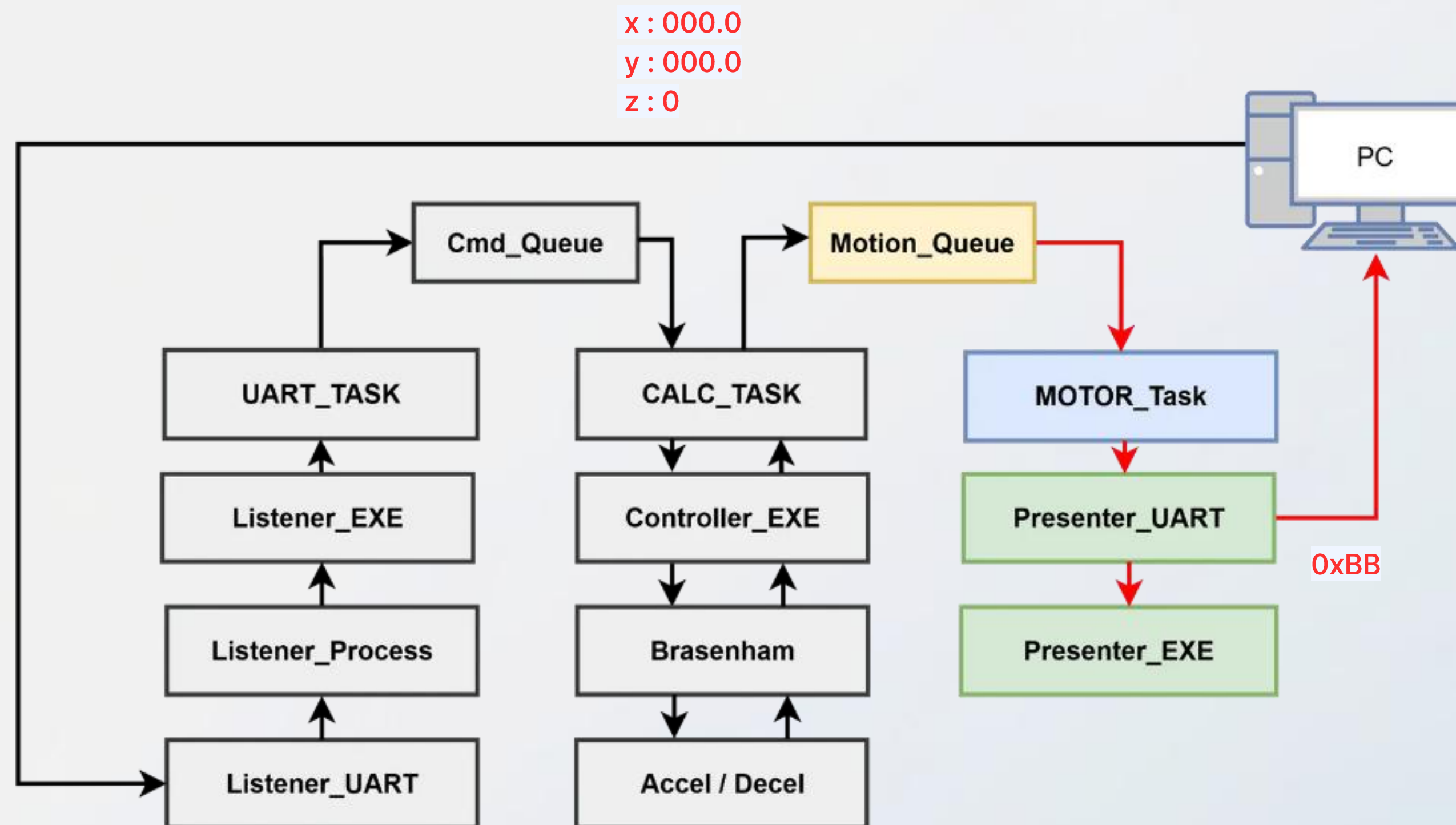


Y 감소

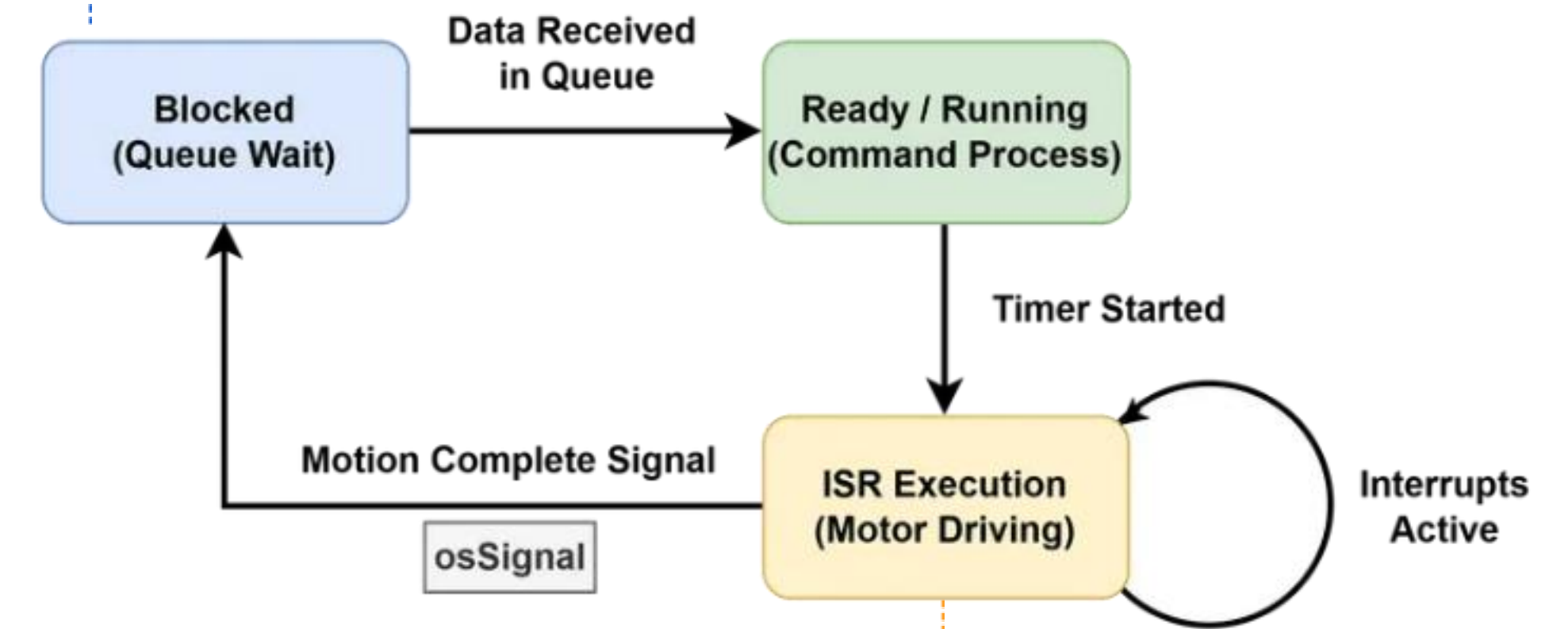


MOTOR_TASK

모터를 실제로 구동시키는 Task



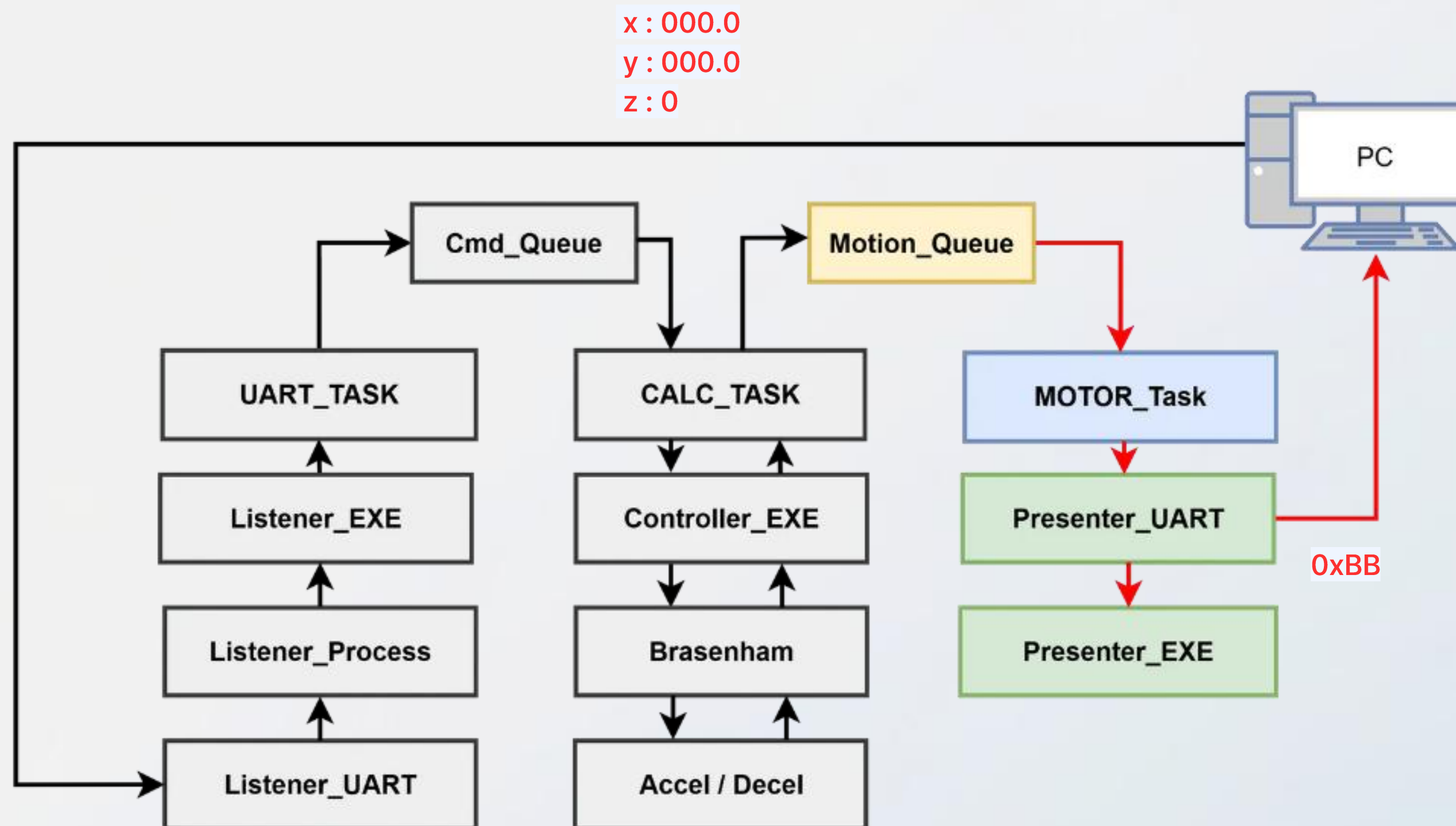
명령 대기 중 Blocked 상태 유지 (CPU 점유율 0%)
이벤트(Queue) 발생 시에만 활성화



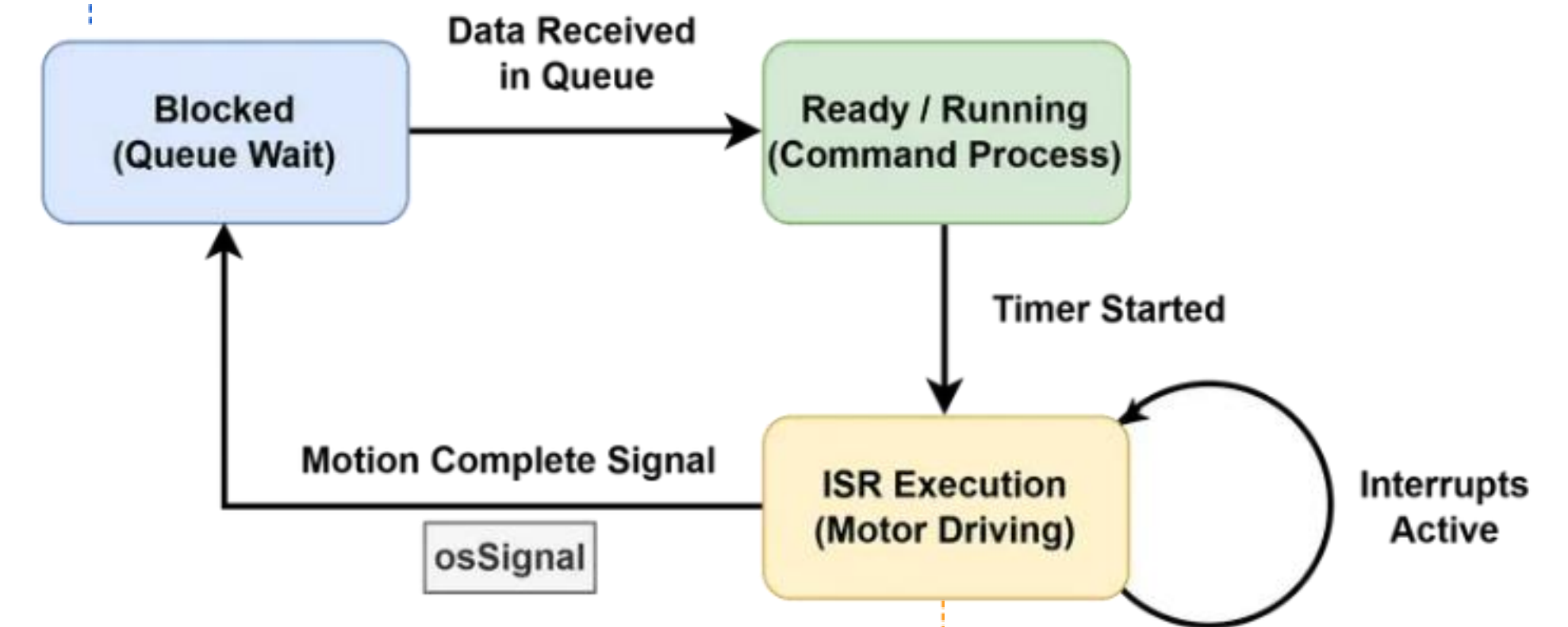
핵심 구동부는 타이머 ISR이 전담 실행
OS 간섭 없는 하드웨어 타이밍 보장

MOTOR_TASK

모터를 실제로 구동시키는 Task



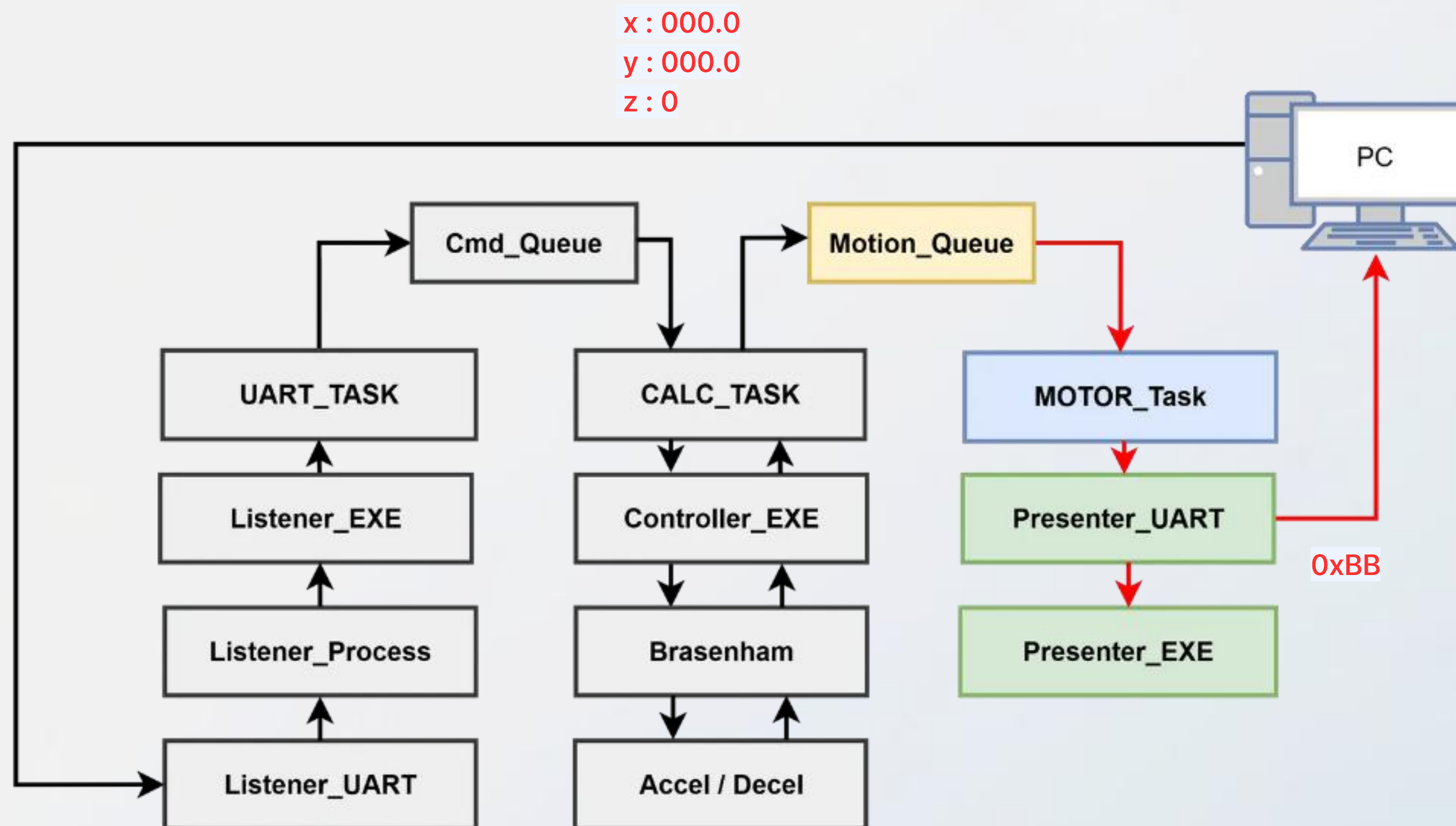
명령 대기 중 Blocked 상태 유지 (CPU 점유율 0%)
이벤트(Queue) 발생 시에만 활성화



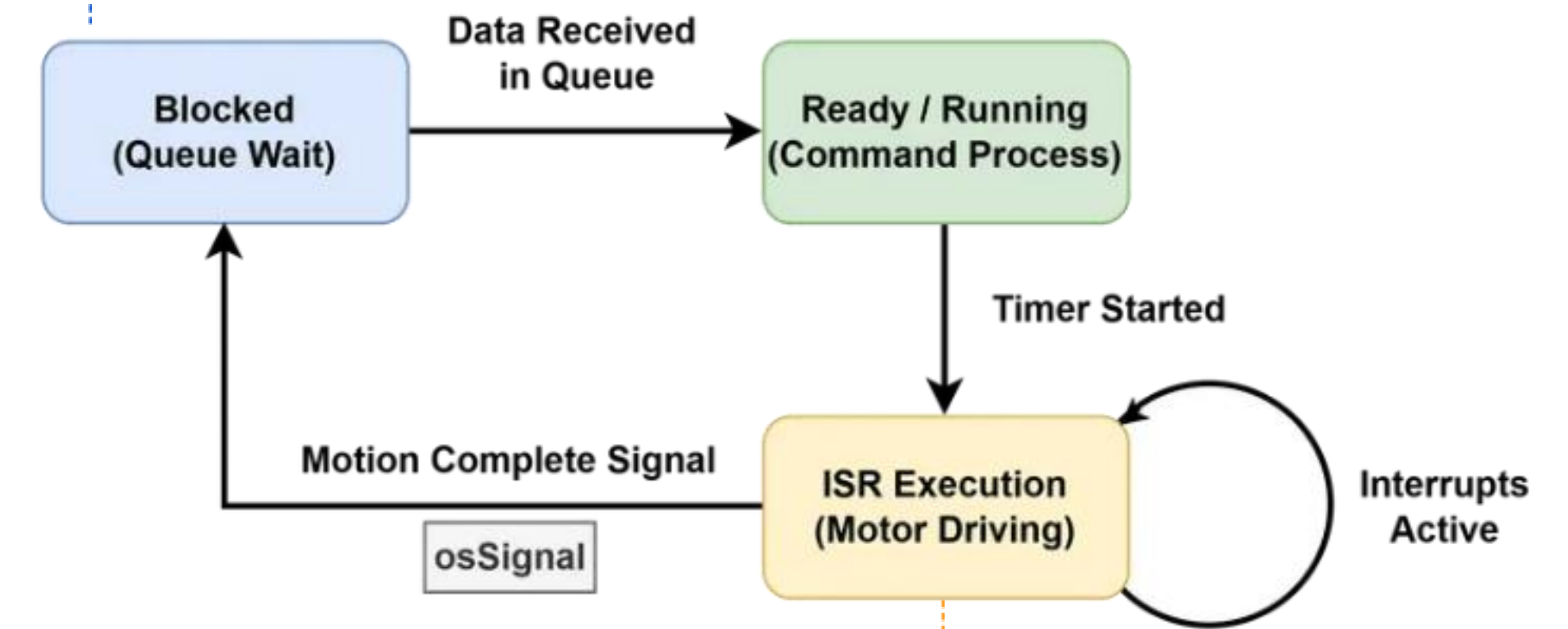
핵심 구동부는 타이머 ISR이 전담 실행
OS 간섭 없는 하드웨어 타이밍 보장

MOTOR_TASK

모터를 실제로 구동시키는 Task



명령 대기 중 Blocked 상태 유지 (CPU 점유율 0%)
이벤트(Queue) 발생 시에만 활성화



핵심 구동부는 타이머 ISR이 전담 실행
OS 간섭 없는 하드웨어 타이밍 보장

05. Troubleshooting



Troubleshooting

Gradient 연산 복잡도에 따른 엣지 품질 저하 문제

[Problem]

- 전처리된 이미지의 품질 저하 현상



원본 이미지



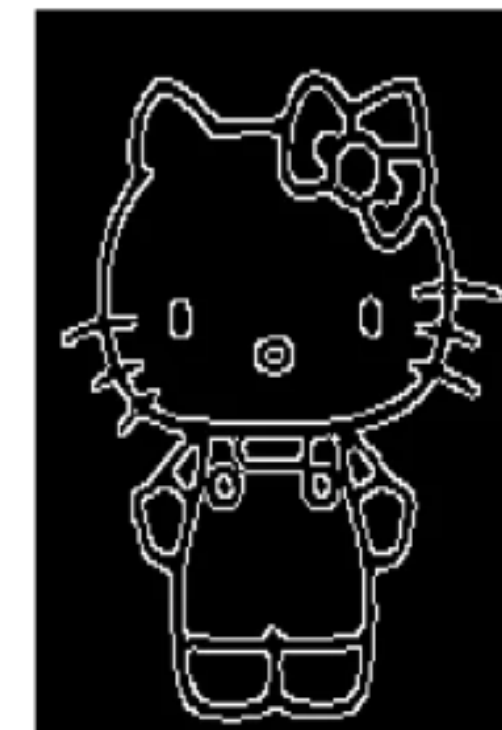
Error Result

[원인 분석]

1. ArcTan2 및 부동소수점 연산의 한계
 - FPGA에서 DSP 자원 과다 소모 및 연산 지연 발생
2. 각도 양자화 오차
 - 정밀하지 못한 각도 분류로 인해 NMS 단계에서 왜곡 발생

[Solution]

- 비교 연산 기반의 방향 최적화 설계



Clear Result

[설계 변경 및 성과]

1. 리소스 최적화
 - 삼각함수 로직 제거, 비트 연산자만으로 판별 로직 재설계
2. 품질 개선
 - 비교 임계값을 적용하여 연산 속도 향상

Troubleshooting

불필요한 좌표 데이터 과다 생성

[Problem]

- 단순화 과정 없는 세분화로 인한 데이터량 증가

```
===== Summary =====
Contours kept: 39
Pen-up distance before optimization (px): 1913.33
Pen-up distance after optimization (px): 1913.33
=====
points before simplify: 26381
points after  simplify: 26381
reduction:  0.0%
=====
Total STM commands: 21674
=====
```

[원인 분석]

1. RDP 단순화 없이 Greedy 이후 Densify를 바로 진행
2. 직선 구간임에도 많은 점 개수 생성 → UART 전송량 증가

[Solution]

- RDP 알고리즘 적용 및 직선 구간 데이터 최적화

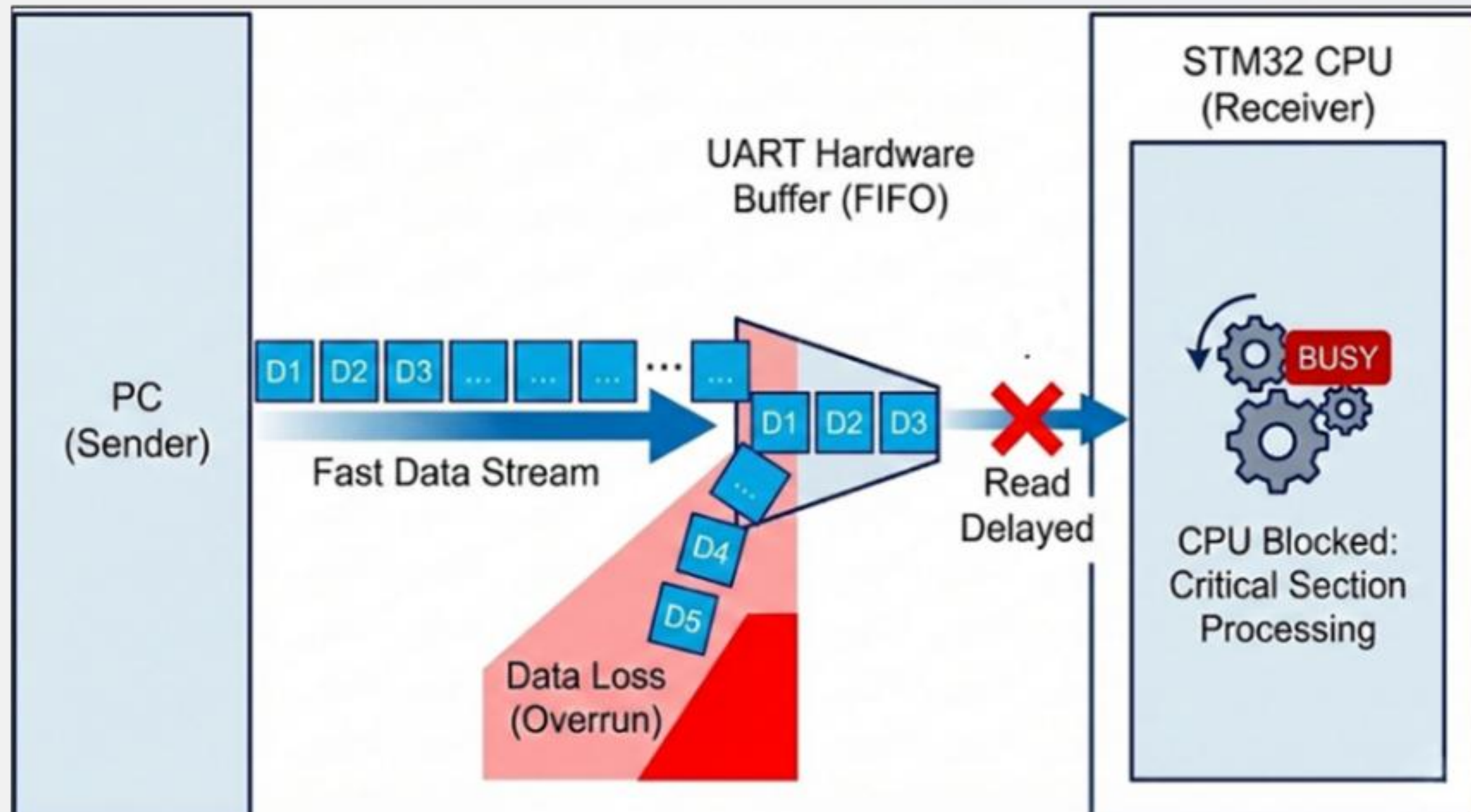
```
===== Summary =====
Contours kept: 39
Pen-up distance before optimization (px): 1913.33
Pen-up distance after optimization (px): 1081.12
=====
points before simplify: 26381
points after  simplify: 21595
reduction: 18.141844509305937 %
=====
Total STM commands: 21674
=====
```

[설계 변경 및 성과]

1. Ramer-Douglas-Peucker(RDP)로 직선 구간 점 제거
2. epsilon_mm 기반으로 형태 오차를 제한하면서 점 개수 감소

Troubleshooting

과도한 Burst 전송으로 인한 인터럽트 처리 불가능



[Problem]

과도한 Burst 전송으로 인한 인터럽트 처리 불능

[기존 로직]

PC에서 64개의 데이터를 Burst Transmit

[발생 결함]

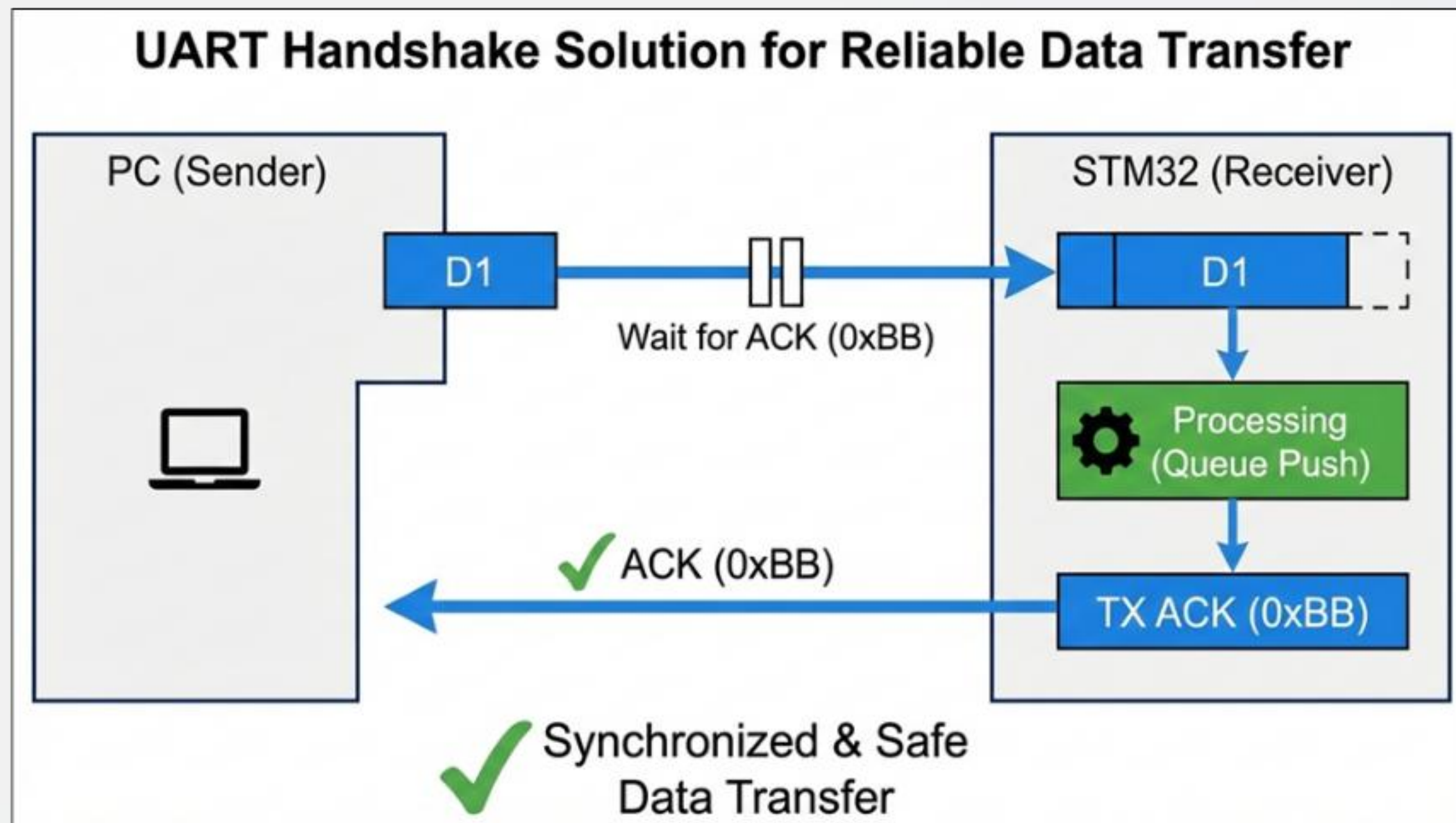
STM32가 첫 데이터를 받아 파싱 → 연산 → motion_queue 적재를 수행하는 동안에도 PC는 계속 데이터를 전송함

[원인 분석]

데이터 처리 중 발생하는 CPU 점유 및 메모리 동시 접근 보호로 인해 차기 UART 인터럽트 서비스가 지연됨

Troubleshooting

과도한 Burst 전송으로 인한 인터럽트 처리 불가능



[Solution]

수요 기반(Demand-driven) 1-to-1 핸드셰이크 도입

[설계 변경]

- 무조건적인 수신 방식에서 "준비 완료 시 요청" 방식으로 전환
- motion_queue에 안전하게 적재 성공 시점에만 ACK 전송
- PC는 ACK 수신 후에만 다음 데이터를 송신하여 전송-처리 주기를 일치시킴

[성 과]

입력-연산-출력 간의 자원 경합을 원천 차단하고,
장시간 구동에도 데이터 무결성 100% 확보

THANK YOU

개인 고찰



변지인 (팀장)

- Camera, UART 등 서로 다른 IP를 하나의 시스템으로 통합하며 모듈 간 인터페이스 규격 정의와 데이터 경로 최적화의 중요성을 체감했습니다. 단순히 개별 로직을 설계하는 것을 넘어 각 모듈이 어떻게 상호작용하는지 이해하는 시스템 레벨 사고를 키울 수 있었습니다.
- 10인 규모 프로젝트를 효율적으로 운영하기 위해 팀원별 기술적 강점을 분석하여 적재적소에 역할을 배치했습니다. Git과 Notion 기반 형상 관리 체계를 구축하여 다수의 개발 결과물을 병목 없이 통합하며 팀 전체의 생산성을 극대화했습니다.
- 개별 로직의 완성도만큼 작업물이 유기적으로 맞물리는 '조율'과 팀원 간 신뢰가 시스템 성패를 결정함을 깨달았으며, 이는 향후 복잡한 설계를 주도하는 경쟁력이 될 것입니다.



채민석

- 이미지 데이터를 Grayscale, Gaussian, Sobel, Canny 4단계 필터로 가공하는 파이프라인을 구축하며 각 연산 단계에서 데이터 손실을 최소화하고 정밀한 엣지를 추출하는 하드웨어 설계 능력을 키울 수 있었습니다.
- 특히 Canny Edge 알고리즘은 보다 복잡한 연산을 수행함에 따라 다양한 처리 기법에 대해서 공부를 하게 되었고 여러라인을 거치면서 계산하기 때문에 라인버퍼와 연산에 대해 고려하여 설계하는 능력을 키울 수 있었습니다.
- 단순한 기능 구현을 넘어 프로젝트의 전반적인 진행 과정을 기록하고 관련 자료들을 꼼꼼하게 정리함으로써 팀의 결과물을 체계적으로 남기는 데 기여했습니다. 이번 경험을 통해 개별 로직의 완성도뿐만 아니라 동료와의 규격 조율, 그리고 철저한 기록이 프로젝트의 전체적인 완성도를 높인다는 것을 깨닫는 소중한 시간이 되었습니다.

개인 고찰



김태형

- 4단계 필터링 과정을 거쳐 GrayScale부터 Canny Edge까지 이어지는 복잡한 영상처리 파이프라인 데이터 구조를 설계하였습니다. 단순히 모듈 각각의 설계 뿐만 아니라 통합과정에서의 디버깅과 검증의 중요성 또한 느낄 수 있는 소중한 경험이었습니다.
- 프로젝트 전 과정에서 환경 설정부터 알고리즘 튜닝 내역, 그리고 통신 딜레이 제어와 같은 Trouble Shooting 과정을 세부적으로 기록하고, 팀원들과의 회의를 통해 시스템 간 인터페이스 문제를 해결해 나간 것이 팀 프로젝트의 체계적인 수행 능력을 기르게 된 소중한 경험이었습니다.



안유한

- 이론으로만 학습했던 RTOS의 Task 관리와 Queue 통신 구조를 실제 시스템에 적용하여 임베디드 설계 원리를 체득했습니다. 그리고 UART로 유입되는 데이터를 Message Queue에 저장하여 이미지 파싱 및 모터 구동 Task간의 흐름을 동기화했습니다. 또한, GUI 설계 과정에서 FPGA, STM32 시스템 간 시리얼 통신시 딜레이 제어를 구현하여 안정적인 통신을 구현하였습니다.
- 프로젝트 전 과정에서 환경 설정부터 일정까지 세부적으로 기록하고, 많은 회의를 통해 문제를 해결해 나간 것이 팀프로젝트의 체계적인 수행 능력을 기르게 된 경험이었습니다.

개인 고찰



이 윤 지

- 프로젝트에서 STM32 환경에서 RTOS 구조를 적용해 모터를 구동하고, 펜 플로터 하드웨어를 직접 조립해 실제 동작까지 구현했습니다. RTOS 기반 코드를 분석하며 태스크들이 큐와 메시지를 통해 서로 데이터를 주고받고, 통로를 통해 동작이 연결되는 구조를 이해했습니다. 또한 펜 플로터를 실제 하드웨어로 조립하고 모터를 연동하면서 하드웨어 연결과 타이밍에 따라 의도하지 않은 동작이 발생할 수 있다는 점을 직접 경험했습니다.
- 이후 태스크 기반으로 모터 제어 기능을 관리하니 수정이나 디버깅도 훨씬 수월해 짐을 느꼈습니다. 또한 하드웨어 조립과 연동은 혼자서 해결하기 어려운 작업이었기 때문에 팀원들과 지속적으로 논의하며 문제를 해결해 나가면서, 시스템 구현에서는 코드의 개별 기능을 완성하는 것뿐만 아니라, 팀원 간 긴밀한 협력이 필수적이라는 것을 알게 되었습니다.



채 준 희

- 평소 관심 있던 주제였던 만큼 플로터의 기구부 제작부터 코드 작성까지 전 과정을 직접 수행하며 프로젝트에 깊이 몰입했습니다. 임베디드 시스템에 RTOS Framework를 도입하여 기능별 태스크 분리와 표준화된 인터페이스를 구축하였고, 이를 기반으로 팀원들과 긴밀하게 소통하며 프로젝트를 진행했습니다.
- 모듈 간의 결합도를 낮춘 덕분에 각 팀원이 코드 간섭 없이 독립적으로 개발에 집중할 수 있었으며, 결과적으로 프로젝트 통합 과정의 소요 시간을 획기적으로 단축하여 협업의 시너지를 극대화할 수 있었습니다

개인 고찰



한규혁

- RTOS 구조 설계와 Queue를 활용한 데이터 흐름, 그리고 UART 통신 등 이론으로 학습한 내용을 실제 설계에 적용하며 확실하게 체득할 수 있는 계기가 되었습니다. 특히 Listener와 Controller의 명확한 기능 분업을 통해 설계 과정이 한층 수월했습니다. PWM과 TIMER를 이용한 정밀한 모터 제어, 그리고 외부 인터럽트를 활용한 실시간 이벤트 처리까지 직접 구현해보며 임베디드 하드웨어 제어에 대한 깊이 있는 이해를 할 수 있었습니다.
- 공유 게시판을 통해 각자의 진행 상황과 마감 일자 등 세부 사항을 공유함으로써 프로젝트 진행이 매우 원활했습니다. 잦은 회의를 통해 부족한 부분과 앞뒤 과정을 지속적으로 점검한 결과 완성도를 높일 수 있었고, 이를 통해 프로젝트 수행에 있어 팀원 간의 소통이 얼마나 중요한지를 다시 한번 깨닫게 되었습니다.